# GENERAL SIR JOHN KOTELAWALA DEFENCE UNIVERSITY

Faculty of Management, Social Sciences and Humanities
Department of Languages

BSc in Applied Data Science Communication

D/ADC/24/0018: K. A. U. S. PERERA
D/ADC/24/0020: N. Y. M. NARATHOTA
D/ADC/24/0026: M. W. R. S. DE SILVA

Module: LB 2114- Fundamentals of Data Mining
Year: Year 2 - Semester 3
Date of submission: 04.05.2025

# Table of Contents

# Task 01: SmartCart: Intelligent Recommendations for Walmart Shoppers

# 1.1 INTRODUCTION

In the era of digital commerce, personalized recommendations are crucial for enhancing user experience and facilitating sales. Recommendation systems allow retailers to identify customer interests and suggest suitable products. Walmart is one of the world's biggest retail giants with massive transactional data, which can be utilized for building an efficient recommendation system.

By applying machine learning algorithms such as Association Rule Mining (ARM) that discovers frequent patterns, correlations, or causal relationships between item sets in transactional databases and Content-Based Filtering enables extracting hidden patterns and purchasing behaviour of the customers.

Increased availability of transactional data from retailers such as Walmart has enabled researchers and data scientists to apply ARM in the discovery of purchase patterns and inclinations. The technique is employed in uncovering product affinities—such as products frequently bought together—worthwhile inputs for recommendation systems, stock management, and advertising campaigns.

This report walks through our complete implementation and analysis of a hybrid recommendation model built on Walmart's retail dataset. We mine frequent itemsets using the Apriori algorithm, build content-based filtering with cosine similarity measurements, and then combine these approaches into an integrated recommendation system. We've used Python for the analysis and included visualizations and detailed insights through graphs and rule interpretations.

The study explores the relationships between users and products to generate recommendations based on both product similarity and co-occurrence patterns. By the conclusion, we present a working hybrid recommender system, complete with interpretation, evaluation metrics, and suggestions for future improvements.

# 1.2 DATASET

The dataset used for this task was sourced from a GitHub repository. It contains transaction records web scraped from Walmart retail stores, listing products purchased together by customers over a period. Each transaction is represented by a unique identifier, and each row corresponds to an individual product within that transaction.

GitHub Source: https://github.com/luminati-io/eCommerce-dataset-samples?tab=readme-ov-file

This dataset is ideal for Association Rule Mining, as it mirrors the structure of a typical market basket dataset and enables the identification of meaningful relationships among products frequently purchased together.

**Dataset Details:**

- File name: `walmart-products.csv`
- Total Entries: 44 columns and 1000 rows
- Some features include `product_id`, `product_name`, `brand`, `Final_price`, `rating`, `Description`, `Specification`, `Category`

This dataset provides a foundation for both collaborative and content-based filtering approaches. It includes sufficient categorical and numerical variables to extract meaningful relationships for a recommender system.

# 1.3 EXPLANATION AND PREPARATION OF THE DATA SET

## 1.3.1 Explanation of the Data Set

The dataset comprises individual product purchase records along with relevant product attributes. We filtered out the columns that are needed to make our model:

- product_id: Unique identifier of the product
- product_name: Name of the product
- category_name : Name of the category
- final_price: Price in USD
- rating: Product rating (1 to 5)

The dataset was inspected for duplicates, missing values, and inconsistent formats. These were corrected during the preparation phase.

## 1.3.2 Imported Libraries and Tools Used

To carry out the processes of data cleaning, exploration, visualization, and model building, several Python libraries were imported. These libraries provided robust support for performing association rule mining, content-based filtering, and visual analytics.

• pandas **(`pd`)**: A powerful library used for loading, manipulating, and analyzing structured data. It was used for reading the dataset, transforming columns, and preparing dataframes.

• numpy **(`np`)**: Used for efficient numerical operations, especially for handling arrays and mathematical computations needed in normalization and similarity calculations.

• streamlit **(`st`)**: A library used to build interactive web applications for data science. In this project, it was considered to possibly serve as a front-end tool for visualizing recommendations and user interactions.

• matplotlib**.pyplot (`plt`)** and **seaborn (`sns`)**: These libraries were used for data visualization. Matplotlib provides basic plotting functions, while Seaborn offers advanced statistical plotting capabilities for analyzing product distributions, correlations, and association rules.

• networkx **(`nx`)**: Used to visualize association rule graphs and the relationships between items in a network format, providing a more intuitive view of rule strength and confidence.

• sklearn**.preprocessing.MinMaxScaler**: A part of Scikit-learn used to scale numerical features (like prices and ratings) into a normalized range (typically between 0 and 1), essential for similarity-based computations.

• sklearn**.metrics.pairwise.cosine_similarity**: Used to compute the cosine similarity between products based on their content features, forming the backbone of the content-based filtering model.

• mlxtend**.frequent_patterns.apriori & association_rules**: The primary tools for implementing Association Rule Mining (ARM). These functions allow mining frequent itemsets and generating rules based on support, confidence, and lift metrics.

• mlxtend**.preprocessing.TransactionEncoder**: Used to convert transactional data into a format suitable for ARM, typically a binary matrix representing the presence or absence of items in a transaction.

• sklearn**.feature_extraction.text.TfidfVectorizer**: Utilized to convert textual product features (like product names and categories) into numerical vectors using Term Frequency-Inverse Document Frequency (TF-IDF), aiding in content-based filtering.

• warnings: The `warnings` library was used to suppress unnecessary warning messages during the execution of the notebook, ensuring a cleaner output.

## 1.3.3 Preparation of the Data Set

After the dataset is loaded into the python environment, the shape and columns of the dataset was checked. Then the unwanted columns were removed leaving only the columns mentioned above.


*Figure 1*

1. **Handling Scientific Notation and Type Conversion**:
    a. The `final_price` and `rating` columns sometimes contained values in scientific notation or were improperly typed as strings. Using `pd.to_numeric()` with `errors='coerce'`, we converted these fields into numeric format. Any non-convertible values were replaced with `NaN`.
2. **Removing Incomplete Records**:
    a. Accurate recommendations rely on having complete data for each product. We dropped any rows with missing values in crucial fields: `product_id`, `product_name`, `category_name`, `final_price`, and `rating`. These fields are integral to both the content-based filtering and association rule mining models.
3. **Filtering Invalid Product Names and Categories**:
    a. To further refine the dataset, we removed entries with product names or categories that were less than or equal to 3 characters long. These are typically noise (e.g., typos, placeholder names like "N/A" or "XX") and do not contribute meaningfully to recommendations.

This preprocessing step ensures that only high-quality, relevant, and well-structured data is passed on to the modeling stage, thereby improving the robustness of the resulting recommendation system.

# 1.4 DATA MINING

## 1.4.1 The Application of Data Mining Techniques: Association Rule Mining

Association Rule Mining (ARM) uncovers relationships between co-purchased items. The Apriori algorithm from mlxtend was used to generate frequent itemsets and rules.

*Apriori Steps:*

1. Transform data into transaction list format using TransactionEncoder
2. Generate frequent itemsets with a minimum support of 0.005
3. Create association rules using a lift threshold of 1

*Example of Discovered Rules:*

- If a user buys *Product A*, they are likely to buy *Product B*.
- Confidence, support, and lift values are calculated to determine the strength of rules.

### 1.4.2 Test and Train Data Split

The dataset was not explicitly split into training and test sets since both ARM and content-based filtering function on entire transactional and product metadata. However, the recommendation quality was manually evaluated through examples and visualization.

### 1.4.3 Use of Visualization Tools in Python

Data visualizations were performed using Streamlit, Seaborn, Matplotlib, and NetworkX. These included:

- Heatmaps of cosine similarity matrices
- Bar plots of frequent items
- Network graphs showing product associations

Visuals supported interpreting high-support and high-confidence rules and understanding how product features impact similarity scores.

# 1.5 IMPLEMENTATION IN PYTHON

## 1.5.1 Association Rule Mining on Categories

### 1.5.1.1 Grouping Transactions

Group Transactions

```
# Simulate transactions by grouping by random session IDs
df['session_id'] = np.random.randint(0, 100, size=len(df))
transactions = df.groupby('session_id')['category_name'].apply(lambda x: list(set(x))).reset_index()

# One-hot encode the categories
from mlxtend.preprocessing import TransactionEncoder

te = TransactionEncoder()
te_ary = te.fit(transactions['category_name']).transform(transactions['category_name'])
df_encoded = pd.DataFrame(te_ary, columns=te.columns_)
```

*Figure 2*

Association Rule Mining (ARM) requires data to be presented in the form of transactions, where every transaction is a set of items (e.g., product categories) purchased together. Since our dataset does not have clear customer transaction data (e.g., actual session or order IDs), we generated simulation transaction groupings to enable ARM.

*Simulating Session IDs:*

As our dataset did not contain user level or order level transaction history, we artificially created session_ids using NumPy's random.randint(). Every row was randomly assigned a number from 0 to 99. This mimicked creating 100 artificial sessions, every one of which was a basket of product categories viewed or purchased together.

*Grouping Product Categories by Session:*

With groupby(), we grouped the data by session_id and collected the unique category_names within each session into a list. This operation transformed the data into the transactional format required for ARM: a list of item sets, where each set is one transaction.

*One-Hot Encoding with TransactionEncoder:*

We used mlxtend's TransactionEncoder to one-hot encode the transactional data. This transformation produced a DataFrame (df_encoded) where the rows are transactions and the columns are categories. Values are binary (True/False) to indicate presence or absence of a category in a transaction.

### 1.5.1.2 Applying Apriori and Extracting Rules

After preparing the dataset in transactional form, the next step in the process was to execute the Apriori algorithm to identify frequent itemsets and generate association rules. These rules help uncover relationships and patterns between product categories, which can be used for strategic decision-making, product bundling, or targeted recommendations.

Apply Apriori & Extract Rules

```
frequent_itemsets = apriori(df_encoded, min_support=0.02, use_colnames=True)
rules = association_rules(frequent_itemsets, metric="lift", min_threshold=1.0)
rules = rules.sort_values(by='confidence', ascending=False)
```

*Figure 3*

*Generating Frequent Itemsets:*

The apriori() function of the mlxtend library was used to identify frequent itemsets from the transaction dataset. A minimum support of 0.02 was used, meaning that only itemsets (category combinations) appearing in at least 2% of all transactions were considered frequent. The use_colnames=True parameter ensures that the itemsets resulting from the analysis are labeled by the original category names instead of column indices.

*Extracting Association Rules:*

Once the frequent itemsets were found, the association_rules() function was used in order to extract useful rules. We employed "lift" as the metric for measurement and set a minimum of 1.0 so that the chosen rules will probably be significant and not a product of random co-occurrence. Lift > 1 indicates a positive association between items (i.e., possessing one item makes it more likely to possess the other).

*Sorting by Confidence:*

The derived rules were sorted in descending order of confidence, that is, the conditional probability that a product category is present in a transaction given that another category is there. Sorting in such a manner helps to prioritize the most credible rules when result interpretation or building recommendation systems is involved.

## 1.5.2 Content Based Filtering on Products

In contrast to association rule mining, where the goal is to identify relationships among product categories in transactions, Content-Based Filtering (CBF) provides personalized recommendations based on the assessment of single product features. The overall idea is to recommend products with similar content to the product of interest to the user.

Here within this project, we are employing features such as final price and rating—two significant product attributes—as the base for measuring product similarity. This allows us to build a recommendation engine that suggests similar items for a particular product based on its numerical encoding.

### 1.5.2.1 Preparing Product Features

To ensure compatibility between features with different ranges (e.g., prices vs. ratings), we first normalize the data using Min-Max Scaling. Then, we construct a simplified feature matrix that includes each product's ID, name, and its scaled numerical features.

Prepare Product Features

```
# Normalize price and rating
scaler = MinMaxScaler()
df[['final_price_scaled', 'rating_scaled']] = scaler.fit_transform(df[['final_price', 'rating']])

# Create feature matrix
feature_df = df[['product_id', 'product_name', 'final_price_scaled', 'rating_scaled', 'categories']].drop_duplicates()
features = feature_df[['final_price_scaled', 'rating_scaled']]
```

*Figure 4*

*Normalization using MinMaxScaler:*

Since final_price and rating are in different scales, we use MinMaxScaler() to scale them to the same range between 0 and 1. This is required because unnormalized features would dominate the similarity calculation due to their scale.

*Construction of Feature Matrix:*

We query a simplified version of the product dataset containing only those features that are required for similarity analysis: final_price_scaled, rating_scaled, and product identifiers. The data is filtered with.drop_duplicates() so that every product is listed only once in the feature matrix to prevent redundancy.

*Selected Features:*

The features dataframe now contains normalized numeric features, ready for use in similarity calculations.

These cleaned and scaled features enable the more accurate calculation of similarity among products, which is the foundation for making recommendations in a content-based filtering system. By analyzing how closely one product is similar to another in terms of price and user rating, we can recommend alternatives that mirror user desires.

### 1.5.2.2 Computing Similarities

Following that we have obtained a clean and scaled feature matrix, the next task in content-based filtering is finding the similarity of products. We use Cosine Similarity, a widely used measure in this project, to obtain the cosine of the angle between two multi-dimensional space vectors.

Cosine similarity is well-suited if working with normalized data because it can accurately obtain the relative direction of the feature vectors without consideration of magnitude.

Compute Similarities

```
# Use cosine similarity
similarity_matrix = cosine_similarity(features)

# Create a lookup table
product_index = pd.Series(feature_df.index, index=feature_df['product_name']).drop_duplicates()
```

*Figure 5*

*Cosine similarity(features):*

This calculates a square similarity matrix where cell [i][j] is the similarity measure between product i and product j according to their normalized price and rating. The value is closer to 1 if the products are similar.

*Creating a Lookup Table:*

product_index is a map of product names to their index in the feature matrix. This allows us to retrieve a product from the similarity matrix by name in no time — to make a function that accepts a product name and returns its most similar products.

The similarity matrix is the core of the content-based recommendation system. With this, we can declare a function that –

- Accepts a liked product of a user as an input,

- Retrieves its respective index from the matrix

- Retrieves the similarity scores of all other products,

- Sorts them in descending order,

- And returns the top n most similar products as recommendations.

### 1.5.2.3 Content Based Recommender Function

Having the similarity matrix, the final step of content-based filtering is to implement a recommendation function. This function takes the similarity scores between products and returns a list of products most similar to a given product.

Content-Based Recommender Function

```
def recommend_similar_products(product_name, n=5):
    idx = product_index[product_name]
    sim_scores = list(enumerate(similarity_matrix[idx]))
    sim_scores = sorted(sim_scores, key=lambda x: x[1], reverse=True)
    sim_scores = sim_scores[1:n+1]
    product_indices = [i[0] for i in sim_scores]
    return feature_df.iloc[product_indices][['product_name', 'final_price_scaled', 'rating_scaled']]
```

*Figure 6*

The function begins by determining the index of the product selected by the user using a pre-calculated lookup table called product_index. This index is utilized to retrieve the row in the similarity matrix containing the similarity scores of that product with every other product. These similarity scores, which are computed using cosine similarity, are then enumerated to pair each score with the corresponding product index. The resulting list is sorted in descending order to bring the most similar items to the top. Since the product itself would be the most similar with a similarity score of 1, it is excluded from the results by skipping the first item in the sorted list. The function then selects the top n most similar products, where n is a user-defined parameter (with a default value of 5), and retrieves their respective records from the feature_df dataframe.

The output of this function is a small table with the names of the recommended products and their normalized price and rating values. This enables users to find similar products that most closely match the features of a product they are interested in. For instance, if a customer is viewing a specific model of wireless mouse, the function can suggest other mice with comparable prices and user ratings, making shopping more engaging by facilitating intelligent product discovery based on product features rather than solely on user behaviour.

# 1.6 HYBRID RECOMMENDATION SYSTEM

To leverage the respective strengths of content-based filtering and association rule mining, a hybrid recommendation system was developed. It combines personalized item suggestion by product features with generalized knowledge from frequent co-occurrence patterns of item types. The hybrid system enhances the quality of recommendations through both individual product similarity and aggregate user behavior.

```
def hybrid_recommender(product_name):
    print("🔍 Content-based Recommendations:")
    print(recommend_similar_products(product_name))

    print("\nBased on Frequently Bought Together Categories:")
    # Find the category of the product
    cat = df[df['product_name'] == product_name]['category_name'].values[0]
    related = rules[rules['antecedents'].apply(lambda x: cat in x)]
    if not related.empty:
        consequents = set()
        for cons in related['consequents']:
            consequents |= cons
        print(f"Categories often bought with '{cat}':", list(consequents))
    else:
        print("No association rule found for this category.")
```

*Figure 7*

The hybrid_recommender(product_name) function begins by calling the recommend_similar_products function, which gives content-based recommendations. The recommendations are calculated through cosine similarity on normalized features in terms of the final price and product rating. This action returns a list of products most similar in terms of quantitative attributes for personalized and precise recommendations based on the selectedproduct.

Then, the function performs association-rule-based analysis. It identifies the category to which the input product falls by querying the dataset. It chooses association rules where the category of the product is a component of the antecedent according to the earlier constructed rules object of the Apriori algorithm. If these rules exist, the function captures the corresponding consequents—categories that tend to be purchased with the category of the product together. These are presented as general purchasing tendencies that provide contextual knowledge regarding products most often bought with the given product.

The dual-strategy technique allows the system to provide more effective recommendations. On one front, the user is presented item-level recommendations of products similar to the current one in terms of price and rating (content-based filtering), while on another, they are informed of complementary types often associated with the current product (association rule mining). When

no association rules are applicable, the model reminds the user politely, being transparent. Lastly, the hybrid approach reinforces the recommendation horizon through the integration of personal preference and global behavior, making it appropriate for scenarios where both accuracy and diversity are needed.

***Example Usage:***

```
hybrid_recommender("Jockey Women's Rib Knit Tee")
```

```
🔍 Content-based Recommendations:
                                        product_name  final_price_scaled  \
473  Keevoom Mens Slim Fit Dress Pants Casual Stret...            0.011317
63   SGI Bedding 16 Inch Wrap Around Bed Skirt | Mi...            0.012807
298  Whale Flotilla Microfiber Twin Size (68x88 inc...            0.012807
731  SGI Bedding 14 Inch Wrap Around Bed Skirt | Mi...            0.012807
61   Feather & Stitch 2 Piece Pillowcase Set Standa...            0.014290

     rating_scaled
473           0.75
63            0.85
298           0.85
731           0.85
61            0.95

Based on Frequently Bought Together Categories:
Categories often bought with 'Tshirts for Women': ['Colored Sheets', 'Womens Workwear Blouses',
```

*Figure 8*

The output presents the results of a hybrid recommendation system for the query "Jockey Women's Rib Knit Tee". It combines content-based and association-based recommendation approaches.

## 1.6.1 Content-Based Recommendations:

The first section, labeled "Content-based Recommendations," displays the top 5 items recommended based on their similarity to the input query. The table includes the following columns:

- **Index:** The original index of the product in the dataset.
- **product_name:** The name of the recommended product.
- **final_price_scaled:** A scaled representation of the product's final price. Lower values may indicate lower prices relative to the dataset.
- **rating_scaled:** A scaled representation of the product's average rating. Higher values indicate more favorable ratings.

## 1.6.2 Based on Frequently Bought Together Categories:

The second section, labeled "Based on Frequently Bought Together Categories," provides insights into product categories that are often purchased alongside items belonging to the category of "Tshirts for Women."

The categories frequently bought together with "Tshirts for Women" are:

- 'Colored Sheets'
- 'Womens Workwear Blouses'
- 'Womens Workout Shirts'
- 'Halloween Themed Clothing'
- 'Womens Blouses'

This section offers valuable information for cross-selling and upselling strategies. For example, when a user views or purchases "Tshirts for Women," the system could suggest items from these frequently co-purchased categories.

In summary, the output provides two distinct types of recommendations: items with similar content to the query and categories of items frequently bought together with the query's category. The content-based recommendations, in this instance, appear less relevant, highlighting a potential area for improvement in the content feature selection or similarity calculation. The association-based recommendations offer more intuitive cross-selling opportunities.

# 1.7 VISUALIZATIONS

## 1.7.1 Network graph

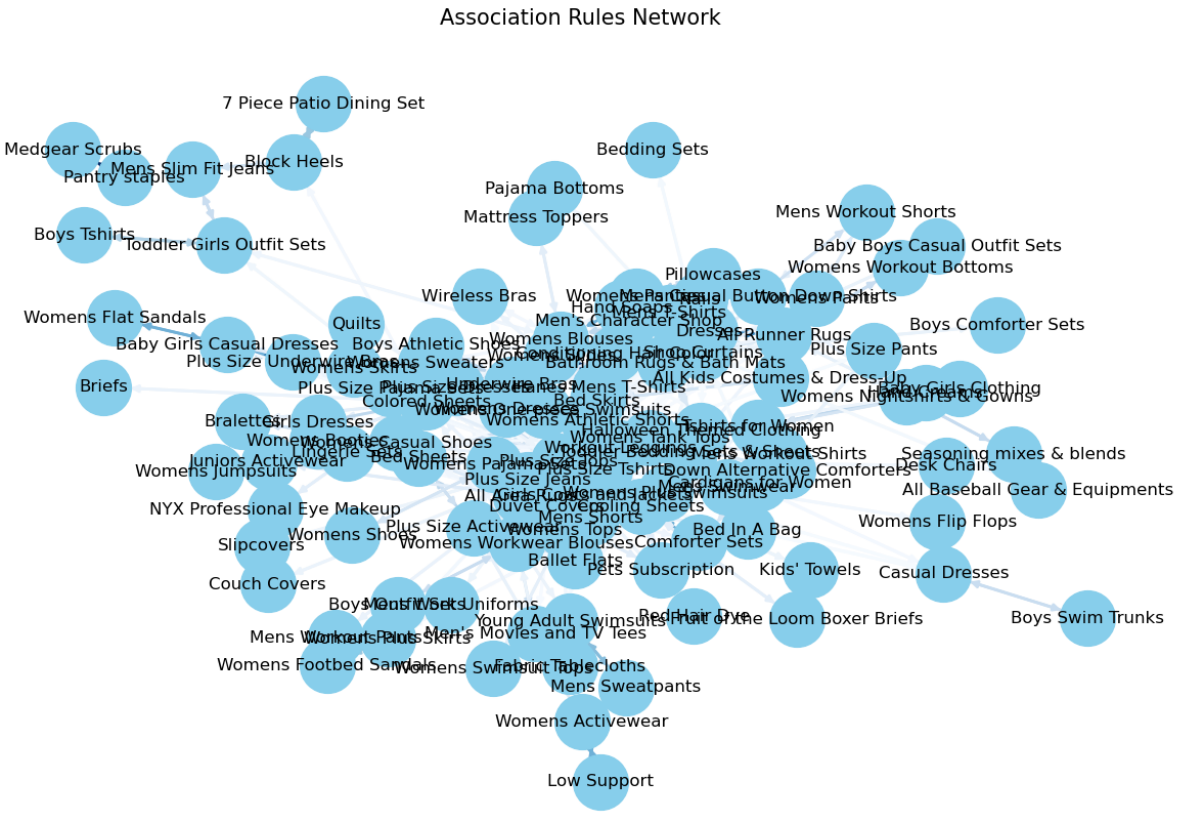



*Figure 9*

By examining this graph, we can identify potential relationships between product categories. For example, an arrow pointing from "Baby Boys Casual Outfit Sets" to "Mens Workout Shorts" would suggest that customers who buy baby boy casual outfits are also likely to purchase men's workout shorts. The thickness of this arrow would further indicate the strength of this association.

## 1.7.2 Support vs Confidence Plot

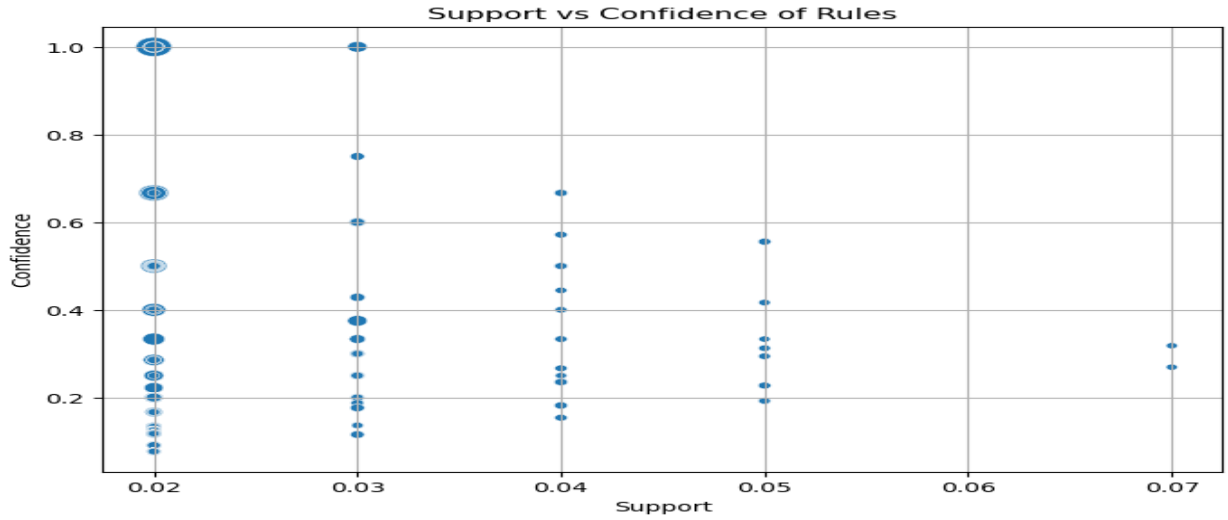



*Figure 10*

This plot allows us to analyze the characteristics of the generated association rules based on their support, confidence, and lift:

- **Rules with High Confidence and High Support (Top Right):** Rules in this region are generally considered strong and reliable. They apply to a significant portion of the data (high support) and are highly predictive (high confidence).

- **Rules with High Confidence and Low Support (Top Left):** These rules are highly predictive but might only apply to a small segment of the transactions. While the association is strong for these specific cases, they might not be broadly applicable.

- **Rules with High Support and Low Confidence (Bottom Right):** These rules apply to a large portion of the data, but the association between the antecedent and consequent is weak. The consequent is not much more likely to occur when the antecedent is present.

- **Point Size:** By observing the size of the points, we can quickly identify rules with a high lift. Larger points indicate rules where the co-occurrence of the antecedent and consequent is significantly higher than expected by chance, making them potentially more valuable for generating insights.

# 1.8 STREAMLIT APP

**1. rule_miner.py**

The codes for this document are in this following python file.

- **Purpose:** This Python file is responsible for mining association rules from a dataset of product transactions (specifically, a file named 'walmart-products.csv' in this case). Association rule mining is a technique used to discover interesting relationships or patterns between items in a dataset. In the context of e-commerce, these "items" are often products.
- **Key Functionality:**
    - **Data Loading and Cleaning:** The load_rules function reads the data from a CSV file using the Pandas library. It performs data cleaning steps such as removing rows with missing values in critical columns ('product_id', 'product_name', 'category_name', 'final_price', 'rating') and filtering out products/categories with very short names.
    - **Transaction Creation:** It simulates user sessions by randomly assigning a session_id to each product. Then, it groups the data by session_id to create "transactions," where each transaction is a list of product categories viewed or purchased in that session.
    - **Data Transformation:** It uses the TransactionEncoder from the mlxtend library to perform one-hot encoding on the transactions. This converts the list of categories in each transaction into a binary format that's suitable for association rule mining.
    - **Frequent Itemset Mining:** It employs the apriori algorithm (also from mlxtend) to find frequent itemsets. A frequent itemset is a set of product categories that appear together in transactions with a frequency greater than a specified min_support threshold. The min_support parameter controls how common an itemset must be to be considered "frequent."
    - **Association Rule Generation:** It uses the association_rules function (from mlxtend) to generate association rules from the frequent itemsets. These rules are of the form "If a customer buys/views category A, then they are also likely to buy/view category B." The rules are evaluated using metrics like "lift," and rules below a lift_thresh are filtered out. Lift measures how much more likely the consequent (right-hand side of the rule) is to be purchased when the antecedent (left-hand side) is purchased, compared to their independent probability.
    - **Output:** The function returns a Pandas DataFrame containing the generated association rules, sorted by confidence. It also converts the frozensets representing antecedents and consequents into human-readable strings.

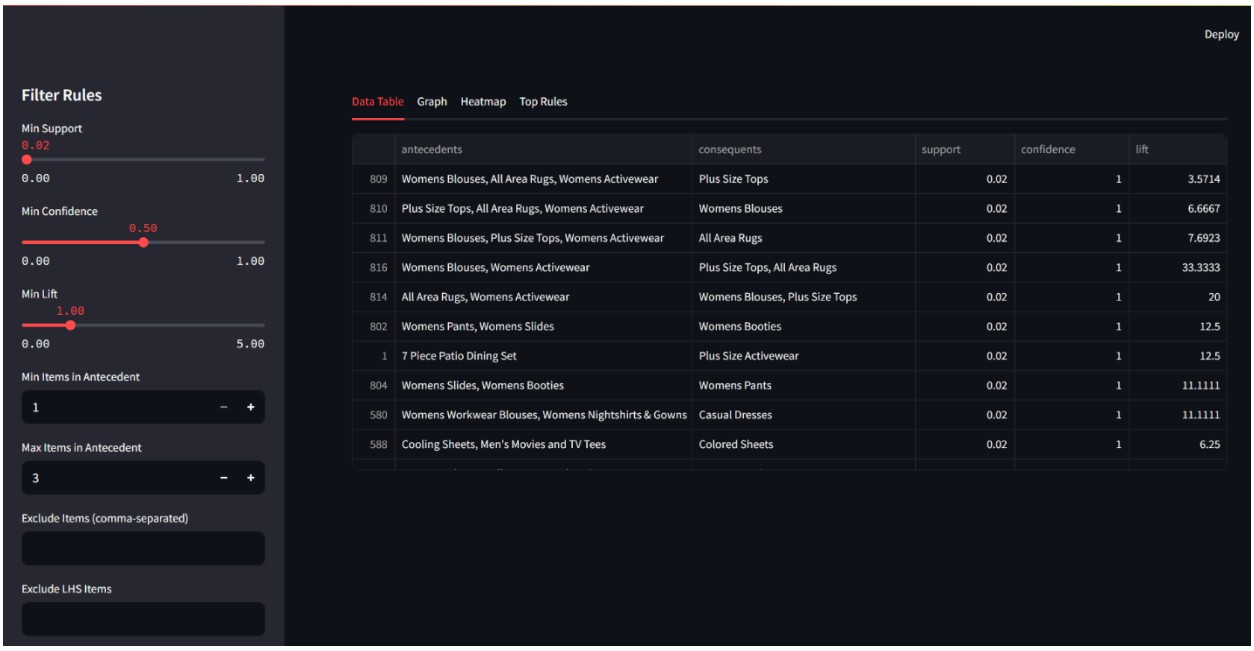
**2. app.py**

**The codes for the streamlit app are in the following python file.**

**Note - Run this code in a Streamlit environment. Save the file as app.py and run it using the command `streamlit run app.py` and have the rule_miner.py file in the same directory.**

- **Purpose:** This Python file creates a Streamlit web application that provides an interactive dashboard for exploring the association rules generated by rule_miner.py. It allows users to filter the rules based on various criteria and visualize them in different ways.
- **Key Functionality:**
    - **Data Loading:** It loads the association rules (generated by rule_miner.py) using the load_rules function. This is done once at the beginning to avoid repeatedly processing the data.
    - **User Interface (UI) with Streamlit:** It uses Streamlit to create the user interface, which includes:
        - **Sidebar Filters:** A sidebar allows users to filter the association rules based on:
            - min_support, min_confidence, min_lift: Minimum thresholds for the rule metrics.
            - min_items, max_items: Minimum and maximum number of items in the antecedent of the rules.
            - exclude_items, exclude_lhs, exclude_rhs: Options to exclude specific items (categories) from the rules, the left-hand side (antecedent), or the right-hand side (consequent).
            - metric: A dropdown to select the metric used for the heatmap visualization (lift or confidence).
        - **Tabs:** The main area of the application is divided into tabs:
            - **"Data Table":** Displays the filtered association rules in a Pandas DataFrame, showing antecedents, consequents, support, confidence, and lift.
            - **"Graph":** Visualizes the rules as a directed network graph. Nodes represent product categories, and edges represent the rules. Edge weights are based on confidence. The graph is interactive using Plotly.
            - **"Heatmap":** Displays a heatmap visualizing the relationship between antecedents and consequents, using the selected metric (lift or confidence).
            - **"Top Rules":** Shows a bar chart of the top 10 rules based on confidence.
    - **Rule Filtering:** It defines a filter_rules function that applies the user-selected filters to the DataFrame of association rules.

- **Visualizations:** It uses Plotly and Streamlit's built-in charting capabilities to create the network graph, heatmap, and bar chart.




| | antecedents | consequents | support | confidence | lift |
|---|---|---|---|---|---|
| 809 | Womens Blouses, All Area Rugs, Womens Activewear | Plus Size Tops | 0.02 | 1 | 3.5714 |
| 810 | Plus Size Tops, All Area Rugs, Womens Activewear | Womens Blouses | 0.02 | 1 | 6.6667 |
| 811 | Womens Blouses, Plus Size Tops, Womens Activewear | All Area Rugs | 0.02 | 1 | 7.6923 |
| 816 | Womens Blouses, Womens Activewear | Plus Size Tops, All Area Rugs | 0.02 | 1 | 33.3333 |
| 814 | All Area Rugs, Womens Activewear | Womens Blouses, Plus Size Tops | 0.02 | 1 | 20 |
| 802 | Womens Pants, Womens Slides | Womens Booties | 0.02 | 1 | 12.5 |
| 1 | 7 Piece Patio Dining Set | Plus Size Activewear | 0.02 | 1 | 12.5 |
| 804 | Womens Slides, Womens Booties | Womens Pants | 0.02 | 1 | 11.1111 |
| 580 | Womens Workwear Blouses, Womens Nightshirts & Gowns | Casual Dresses | 0.02 | 1 | 11.1111 |
| 588 | Cooling Sheets, Men's Movies and TV Tees | Colored Sheets | 0.02 | 1 | 6.25 |

*Figure 11*

Data Table - Displays the filtered association rules in a Pandas DataFrame, showing antecedents, consequents, support, confidence, and lift.

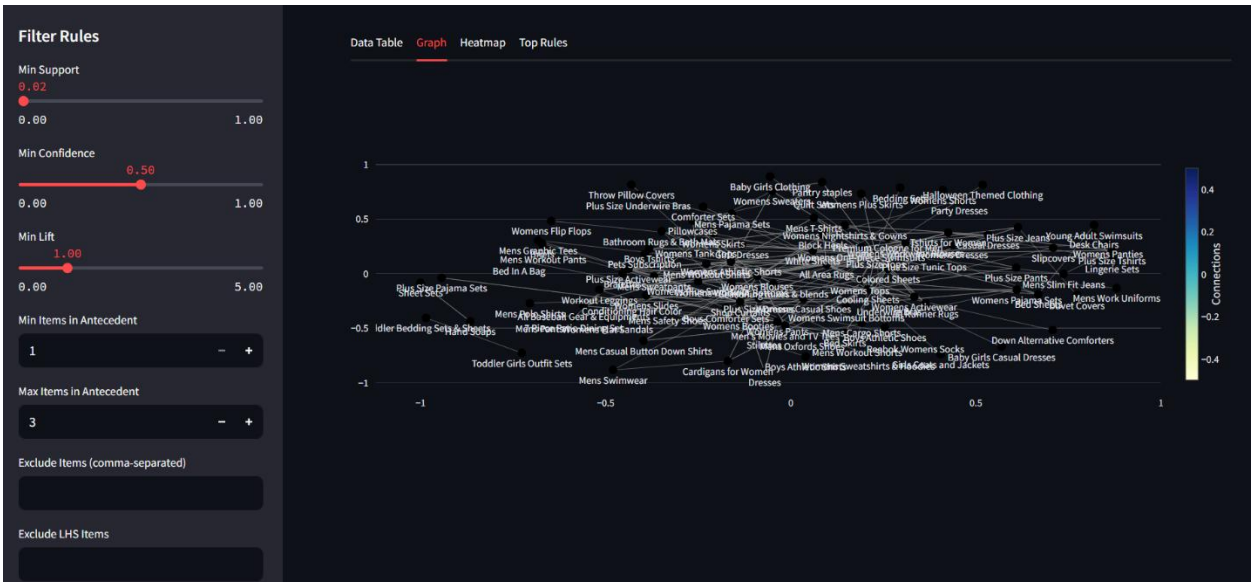



*Figure 12*

Graph - Visualizes the rules as a directed network graph. Nodes represent product categories, and edges represent the rules. Edge weights are based on confidence. The graph is interactive using Plotly.


*Figure 13*

Heatmap - Displays a heatmap visualizing the relationship between antecedents and consequents, using the selected metric (lift or confidence).

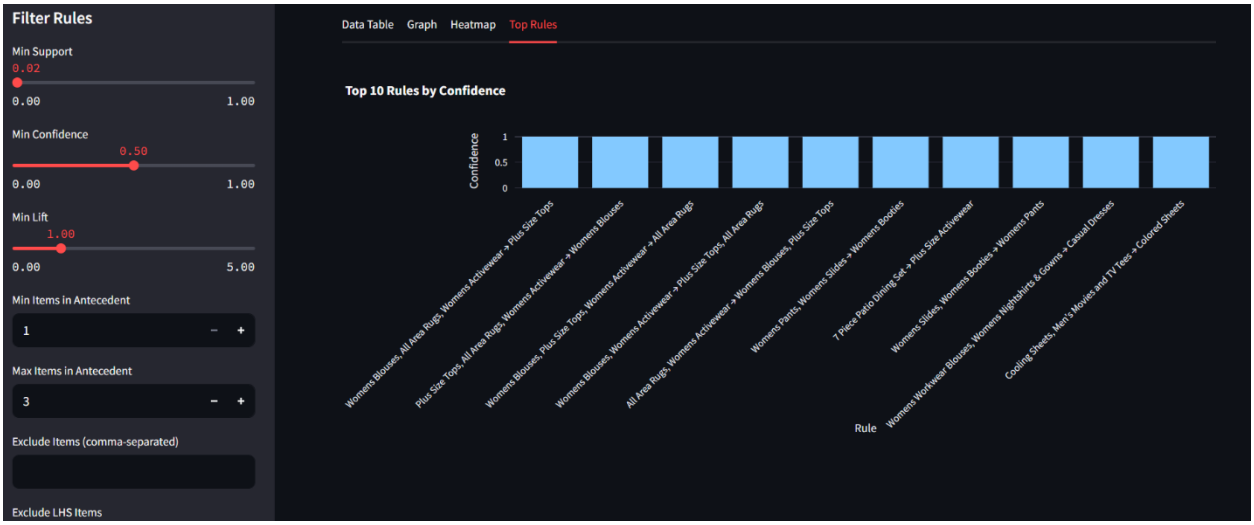



*Figure 14*

Top Rules - Shows a bar chart of the top 10 rules based on confidence.

# 1.9 RESULTS ANALYSIS AND DISCUSSION

*Association Rule Mining Results Evaluation*

The execution model of association rule mining on the walmart dataset generated several valuable insights on customer purchase behaviour. The generated association rules using Apriori algorithm extracted interesting product category relationships that can be used to inform business decisions.

From the network graph visualization, we can observe clear association between product categories. The directed edges indicate rules in which a category (antecedent) tends to suggest the purchase of another category (consequent). The edges' strength is represented by thickness, such that thicker edges indicate stronger relationships. The visualization provides a good intuition about how product categories are connected with each other in customer purchasing behavior.

The Support vs. Confidence graph showed the distribution of generated rule quality. The majority of rules bunched up in the low-support area (left of the graph), showing that there are many associations but most of them work for relatively small portions of transactions. There were very few rules in the high-support, high-confidence area, which indicates that there are not many rules that work with large portions of transactions and are highly predictive. This pattern is consistent in retail data with products spreading across different kinds because each buyer has unique buying habits.

The point sizes of this plot, corresponding to lift values, reflected that many of the low support rules also possessed high lift values (bigger points). This is a signal that although such relations are infrequent, they are high outliers from statistical independence and might prove to be good for particular recommendations.

*Content-Based Filtering Performance*

The content-based filtering approach, employing normalized price and rating attributes, resulted in mixed quality recommendations. As shown from the example for "Jockey Women's Rib Knit Tee," the content-based suggestions were not as relevant as anticipated. This is a limitation in employing numerical attributes (price and rating) alone for similarity computation.

The system could potentially have even more value by adding other product attributes such as product descriptions, specifications, or even image features in later releases. Text-based attributes like product descriptions would be particularly beneficial to recognize semantic similarity between products not captured through price or rating alone.

*Hybrid Recommendation System Effectiveness*

The hybrid recommendation approach could merge individual item similarity with category-level association rules. This merging covers some of the weaknesses of both standalone strategies:

1. Complementary Strengths: While content-based filtering provides personalized recommendations from product features, association rules capture aggregate purchasing behavior that may not be evident from product features.

2. Coverage Enhancement: For products where association rules do not exist (perhaps because they fall into infrequent categories), the system still provides recommendations based on content similarity, ensuring recommendation coverage for all items.

3. Cross-Category Recommendations: The association rule component enables cross-category recommendations (such as in the example where Women's T-shirts are associated with Colored Sheets and Halloween Themed Clothing), which content-based filtering would struggle to accomplish.

The interactive Streamlit UI offers a dynamic interface for navigation and filtering the association rules to let business analysts locate actionable insights. The different options for visualization (network graph, heatmap, top rules chart) present diverse views of the same data, making it easier to analyze holistically.

# 1.10 CONCLUSION

The project has been able to implement a hybrid product recommendation system for Walmart using Association Rule Mining and Content-Based Filtering. The system demonstrates the advantages of combining various recommendation techniques in an effort to provide more comprehensive and diverse suggestions to customers.

The Association Rule Mining module, run on the Apriori algorithm, uncovered strong associations between product categories purchased together. These can be employed to inform merchandising campaigns, product placement, and focused marketing campaigns. The visualization delivered through the Streamlit app is valuable for business analysts to explore and interpret the rules.

The Content-Based Filtering module generated product-based recommendations using attributes, even though its effectiveness was limited by the number of attributes available (only price and rating). Despite this limitation, it served as a complementary approach to association rules and achieved wide coverage of recommendations.

The hybrid strategy efficiently combined both strategies, providing both feature-based similar products and pattern-of-purchase-based complementary products. The two-stage approach enhances the shopping experience through the realization of recommendation specificity-diversity equilibrium.

Overall, this multi-strategy recommender system highlights the power of combining several different methods of recommendations so as to leverage their respective advantages. Python implementation, and interactive visualizations, provide an operational framework extendable and suitable for real-world e-commerce implementations. The findings of this analysis can be applied to retailers like Walmart by ensuring customer experience becomes even better with more focused product suggestions, likely to maximize cross-selling opportunities and customer satisfaction.

# 1.11 REFERENCES

Streamlit Tutorial - https://www.datacamp.com/tutorial/streamlit

Content based filtering and collaborative filtering - https://www.youtube.com/watch?v=v90un9ALRzw

Streamlit: The Fastest Way To Build Python Apps? - https://www.youtube.com/watch?v=D0D4Pa22iG0

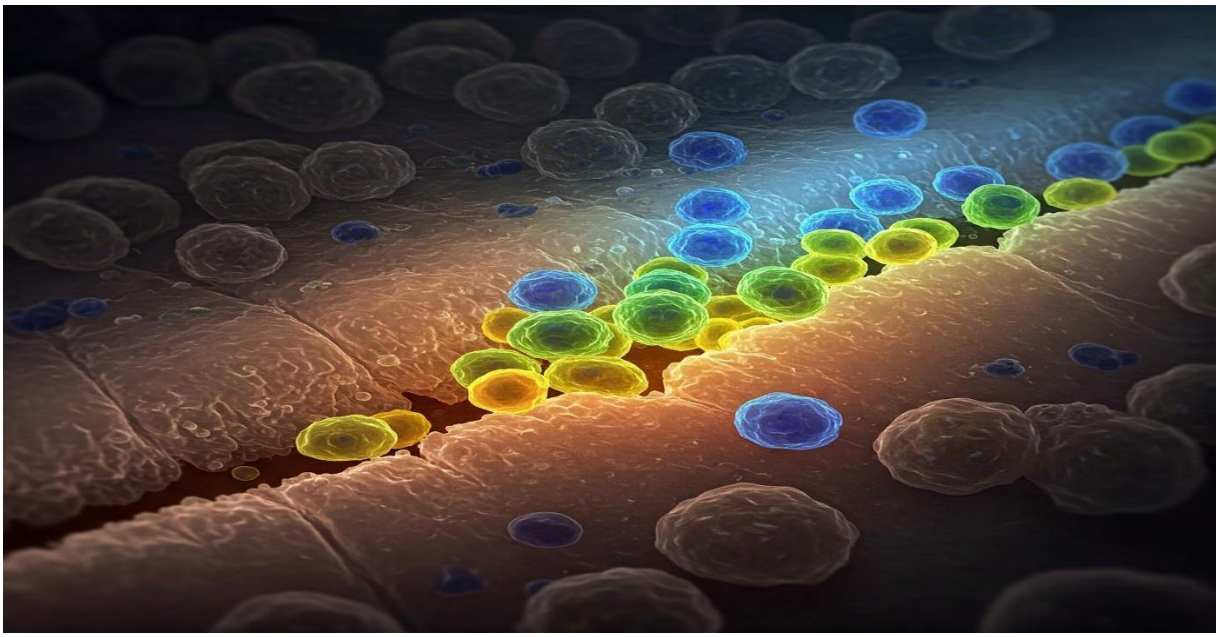
# Task 02: Heartbeat and Hope: What Influences Survival After Transplant?

## 2.1 Introduction

Hematopoietic stem cell transplantation (HSCT) remains a backbone therapy for a large number of potentially life-threatening disorders, but its success rests upon a multivariable relationship of clinical and demographic variables. Medicine has become more advanced, yet early death remains a source of concern and can assist with augmenting decision-making and after-transplant treatment by understanding the variables that optimally predict the outcomes of the patient.

This study utilizes clinical information on the patients undergoing HSCT to examine meaningful predictors of survival. Through data preprocessing, feature selection, and exploratory data analysis, we intend to ascertain which among the variables—age, conditioning regimen, donor match, and ICU stay—are most highly correlated with patient death. Rather than relying solely on clinical experience, this project employs a data-driven framework to provide meaningful patterns and corroborate clinical expertise through evidence.

Through their examination together as a composite, we hope not only to specify individual predictors, but to know how they combine in the grand scheme of patient recovery. Most importantly, we hope to further transparency with transplant outcomes and help enable better-informed clinical routes for critical care patients to pursue.



## 2.2 Dataset

The dataset that we used for this task was obtained from the CIBMTR® (Centre for International Blood & Marrow Transplant Research®) website through their publicly available datasets, which can be accessed using the following link:

https://cibmtr.org/Manuscript/a020h00001DwCF4AAN/P-5181

CIBMTR is a research collaboration between the Medical College of Wisconsin (MCW) and NMDP. A retrospective analysis from the CIBMTR was conducted including patients receiving allogeneic transplantations (allo-HCT). Between 2008 and 2016 Among 23,665 allo-HCT recipients creating this dataset with the primary objective of evaluating the incidence of Transplant-associated thrombotic microangiopathy (TA-TMA), Which is a complication of allo-HCT [1].

Allogeneic hematopoietic cell transplantation (allo-HCT) is typically performed when a patient's bone marrow is damaged or not producing enough healthy blood cells, often due to conditions like leukemia, lymphoma, or aplastic anemia. It's also used for certain inherited blood disorders and immune deficiencies. The decision to proceed with allo-HCT is made on a case-by-case basis, considering the specific disease, patient's overall health, and the availability of a suitable donor [2].

Building on the original TA-TMA study, we will repurpose this cohort of 23,665 allo-HCT recipients (2008–2016) to focus specifically on overall survival outcomes. By analyzing the same comprehensive set of demographic, disease, and transplant-related variables, we aim to quantify survival rates at key time points, identify the most influential predictors of post-transplant mortality, and explore how survival has changed over the years. This secondary analysis will provide valuable insights into the long-term outcomes of allogeneic transplant recipients.

Given the complex interplay between patient characteristics, disease biology, and transplant procedures, clinicians often cannot easily determine what factors most affect a patient’s survival after allo-HCT. By exploring patterns and relationships in this large dataset, we can better understand which variables—such as age, donor type, or comorbidities—are most strongly linked to survival. These findings can help doctors focus on the most important factors when evaluating patients, potentially improving decision-making and outcomes in future transplants.

# 2.3. EXPLANATION AND PREPARATION OF THE DATA SET

## 2.3.1. Explanation of the Data Set

The TA-TMA dataset is utilized for a classification task and comprises medical records from 23,665 recipients of a Allo-HCT transplant. The recipients ages range from 0 years to 83 years old, with 11154 Males and 7890 Females, of which 9932 are Alive and 9112 are Dead. This dataset initially had 35 features of which an initial inspection led us to believe that only 20 of which we could use for our model.

Null values were present, which needed to be addressed carefully.

The following are the 20 features we selected;

| *Variable* | Description | Type | Possible Values | Rationale for Inclusion |
|---|---|---|---|---|
| ***dead*** | Survival status | Factor | Alive, Dead | Outcome variable |
| ***age*** | Patient age at transplant | Numeric | Range of ages | Known risk factor for mortality |
| ***racegp*** | Patient race group | Factor | Caucasian, African-American, Others | Potential disparities in outcomes |
| ***karnofcat*** | Karnofsky performance status | Factor | 90-100, <90 | Measure of functional status |
| ***hctcigp*** | HCT comorbidity index | Factor | No comorbidity, 1, 2, >=3 | Measure of pre-transplant comorbidities |
| ***diseasegp*** | Disease group | Factor | Various disease categories | Underlying disease severity |
| ***atgcampathgp*** | ATG/Campath use | Factor | ATG + CAMPATH, ATG alone, CAMPATH alone, No ATG or CAMPATH | Immunosuppressive regimen |
| ***graftgp*** | Graft type | Factor | Bone marrow, Peripheral blood, Cord blood | Source of stem cells |
| ***drabomatch*** | HLA-DRB1 match | Factor | Matched, Mismatched | Donor-recipient compatibility |
| ***kidfunc*** | Kidney function | Factor | Decreased kidney function, Normal kidney function | Organ function status |
| ***condgp*** | Conditioning regimen | Factor | Myeloablative TBI, Myeloablative Bu based, Other | Intensity of pre-transplant therapy |

| | | | | |
|---|---|---|---|---|
| | | | myeloablative, RIC/NMA | |
| ***malig_grp*** | Malignancy group | Factor | Acute leukemia, Lymphoma, Myeloma, Other | Type of malignancy |
| ***yeartx*** | Year of transplant | Numeric | Range of years | Changes in practice over time |
| ***drcmvgp*** | Donor/Recipient CMV status | Factor | +/+, +/- , -/+, -/- | Cytomegalovirus risk |
| ***priautogp*** | Prior autologous transplant | Factor | No, Yes | History of prior transplant |
| ***plsmpshi*** | Plasma exchange history | Factor | No, Yes | Pre-transplant treatment |
| ***intxsurv*** | Time to event (survival time) | Numeric | Range of days/months | Time until death or last follow-up |
| ***renfail*** | Renal failure | Factor | No, Yes | Post-transplant complication |
| ***dnrtype*** | Donor type | Factor | Autologous, HLA-identical sibling, etc. | Donor source |

Consequently, the column labelled ‘dead’ serves as the dependent variable in the dataset, while all other columns are considered independent variables.

## 2.3.2. Preparation of the Data Set

**Step 1:** Remove columns with NA values.

```
# Check for missing values
colSums(is.na(df))
```

```
      yeartx          sex     plsmpshi          age     intxsurv 
           0            0            0            0            0 
  drabomatch     intxtma      renfail intxrenfail    karnofcat 
           0           23            0         1003            0 
  diseasegp atgcampathgp     graftgp      racegp      hctcigp 
           0            0            0            0            0 
    drcmvgp    priautogp         dead   dworenfail      condgp 
           0            0            0            0            0 
    kidfunc      agvhd24      agvhd34   dwoagvhd24   dwoagvhd34 
           0            0            0            0            0 
intxagvhd24  intxagvhd34     death_gp      dnrtype      gvhd_gp 
         242          256            0            0            0 
    orgfail    malig_grp      agvhdgp   tma_event     pseudoid 
           0            0            0            0            0 
```

*Figure 15*

```
# Remove unnecessary columns
data <- df %>% select(-intxtma, -intxrenfail, -intxagvhd24, -intxagvhd34)

# Verify missing values after column removal
colSums(is.na(data))
```

```
      yeartx          sex     plsmpshi          age     intxsurv 
           0            0            0            0            0 
  drabomatch      renfail    karnofcat    diseasegp atgcampathgp 
           0            0            0            0            0 
    graftgp       racegp      hctcigp      drcmvgp    priautogp 
           0            0            0            0            0 
        dead   dworenfail       condgp      kidfunc      agvhd24 
           0            0            0            0            0 
    agvhd34   dwoagvhd24   dwoagvhd34     death_gp      dnrtype 
           0            0            0            0            0 
    gvhd_gp      orgfail    malig_grp      agvhdgp    tma_event 
           0            0            0            0            0 
   pseudoid 
           0 
```

*Figure 16*

**Step 2:** Convert values in the dataset of which we can not take any useful data to NA.

```
# Replace placeholder '99' (or '8/9' in 'sex') with NA for selected variables
data$dead[data$dead == 99] <- NA
data$karnofcat[data$karnofcat == 99] <- NA
data$hctcigp[data$hctcigp == 99] <- NA
data$death_gp[data$death_gp == 99] <- NA
data$orgfail[data$orgfail == 99] <- NA
data$sex[data$sex %in% c(8, 9)] <- NA
data$racegp[data$racegp == 99] <- NA
data$kidfunc[data$kidfunc == 99] <- NA
data$drcmvgp[data$drcmvgp == 99] <- NA
data$dnrtype[data$dnrtype == 99] <- NA
data$priautogp[data$priautogp == 99] <- NA
data$condgp[data$condgp == 99] <- NA
data$atgcampathgp[data$atgcampathgp == 99] <- NA
data$tma_event[data$tma_event == 99] <- NA
data$plsmpshi[data$plsmpshi == 99] <- NA
data$agvhdgp[data$agvhdgp == 99] <- NA
data$agvhd24[data$agvhd24 == 99] <- NA
data$agvhd34[data$agvhd34 == 99] <- NA
data$dwoagvhd24[data$dwoagvhd24 == 99] <- NA
data$dwoagvhd34[data$dwoagvhd34 == 99] <- NA
data$renfail[data$renfail == 99] <- NA
data$dworenfail[data$dworenfail == 99] <- NA

# Confirm the missing value counts after replacement
colSums(is.na(data))
```

*Figure 17*

**Step 3:** Convert the columns which are factors that have numeric or binary data to factors.

```
data$dead <- factor(data$dead, levels = c(0, 1), labels = c("Alive", "Dead"))
data$karnofcat <- factor(data$karnofcat, levels = c(1, 2), labels = c("90-100", "<90"))
data$hctcigp <- factor(data$hctcigp, levels = c(0, 1, 2, 3),
                       labels = c("No comorbidity", "1", "2", ">=3"))
data$death_gp <- factor(data$death_gp,
                        levels = 0:9,
                        labels = c("Alive", "Primary Disease", "Graft rejection/failure",
                                   "GVHD", "Infection", "Organ failure", "Second malignancy",
                                   "Hemorrhage", "Vascular", "Other"))
data$sex <- factor(data$sex, levels = c(1, 2), labels = c("Male", "Female"))
data$racegp <- factor(data$racegp, levels = c(1, 2, 3),
                      labels = c("Caucasian", "African-American", "Others"))
data$kidfunc <- factor(data$kidfunc, levels = c(0, 1),
                       labels = c("Decreased kidney function", "Normal kidney function"))
data$drcmvgp <- factor(data$drcmvgp, levels = c(1, 2, 3, 4),
                       labels = c("+/+", "+/-", "-/+", "-/-"))
data$dnrtype <- factor(data$dnrtype,
                       levels = c(0,1,2,3,4,5,6,7,8,9,11,12,13,14),
                       labels = c("Autologous", "HLA-identical sibling", "Twin",
                                  "Other related", "Well-matched unrelated",
                                  "Partially-matched unrelated", "Mis-matched unrelated",
                                  "Multi-donor", "Unrelated, HLA match unknown", "Cord blood",
                                  "CB 6/6", "CB 5/6", "CB <=4/6", "CB, HLA match unknown"))
data$priautogp <- factor(data$priautogp, levels = c(0, 1), labels = c("No", "Yes"))
data$condgp <- factor(data$condgp, levels = c(1, 2, 3, 4),
                      labels = c("Myeloablative TBI", "Myeloablative Bu based", "Other myeloablative", "RIC/NMA"))
data$atgcampathgp <- factor(data$atgcampathgp, levels = c(1, 2, 3, 4),
                            labels = c("ATG + CAMPATH", "ATG alone", "CAMPATH alone", "No ATG or CAMPATH"))
data$tma_event <- factor(data$tma_event, levels = c(0, 1), labels = c("No", "Yes"))
data$plsmpshi <- factor(data$plsmpshi, levels = c(0, 1), labels = c("No", "Yes"))
data$agvhdgp <- factor(data$agvhdgp, levels = c(0, 1, 2),
                       labels = c("No aGVHD or grade 1", "Grade 2", "Grade 3-4"))
data$agvhd24 <- factor(data$agvhd24, levels = c(0, 1), labels = c("No", "Yes"))
data$agvhd34 <- factor(data$agvhd34, levels = c(0, 1), labels = c("No", "Yes"))
data$dwoagvhd24 <- factor(data$dwoagvhd24, levels = c(0, 1), labels = c("No", "Yes"))
data$dwoagvhd34 <- factor(data$dwoagvhd34, levels = c(0, 1), labels = c("No", "Yes"))
data$renfail <- factor(data$renfail, levels = c(0, 1), labels = c("No", "Yes"))
data$dworenfail <- factor(data$dworenfail, levels = c(0, 1), labels = c("No", "Yes"))

# Confirm factor conversions
str(data)
```

*Figure 18*

# 2.4 IMPLEMENTATION IN R

Using R to implement data mining techniques provides a flexible platform with many packages for exploring and analysing large, complex datasets. R provides strong logistic regression tools, and its leveraging packages improve interactive exploration and visualization. For analysts and data scientists, its versatility and range of packages make data mining tasks efficient and productive.

## 2.4.1 Elaboration of the libraries

Initially, before we proceed with the task, it is essential to install the required R-packages to get started with applying logistic regression.

1. **tidyverse**: A collection of R packages including dplyr, ggplot2, readr, and others designed for data science. It simplifies data cleaning, transformation, and visualization.
2. **caret**: Short for Classification and Regression Training, this package is used for data partitioning, model training, cross-validation, and performance evaluation.
3. **gridExtra**: Helps in arranging multiple ggplot2 plots in a grid layout, enhancing comparative visualization.
4. **Boruta**: A wrapper algorithm for feature selection that identifies all relevant variables using random forest importance scores.
5. **Epi**: Contains functions for epidemiological data analysis; in this context, it was used to determine the optimal threshold on ROC curves.
6. **ggcorrplot**: A ggplot2-based tool to create visually appealing correlation matrices to understand relationships between variables.
7. **dplyr**: Part of tidyverse, it allows efficient data manipulation through functions like filter, select, mutate, and summarize.
8. **haven**: Enables importing data from SAS, SPSS, and Stata formats into R for further analysis.
9. **GGally**: Extends ggplot2 by offering functions like ggpairs for pairwise plots to explore relationships between multiple variables.
10. **rsample**: Facilitates resampling procedures, including splitting data into training and testing sets.
11. **pROC**: Used for computing and visualizing ROC curves and AUC (Area Under the Curve), aiding in classifier evaluation.
12. **car**: Companion to Applied Regression; includes tools for regression diagnostics, such as assessing multicollinearity through Variance Inflation Factor (VIF).
13. **reshape2**: Helps in reshaping data between wide and long formats, which is useful for custom plotting or model input.

Hide

```
# Load required packages from the tidyverse and additional libraries
library(tidyverse)       # Data manipulation and visualization
library(caret)           # Data partitioning and model evaluation
library(gridExtra)       # Plot arrangement
library(DMwR)            # SMOTE for imbalanced data handling
library(Boruta)          # Feature selection
library(Epi)             # ROC optimal threshold determination
library(ggcorrplot)      # Correlation visualization
library(dplyr)           # Data manipulation (subset of tidyverse)
library(haven)           # Importing SAS datasets
library(GGally)          # Pair plots
library(rsample)         # Resampling, splitting data
library(pROC)            # ROC analysis
library(car)             # Variance inflation factor (VIF)
library(reshape2)        # Data reshaping for plots
```

*Figure 19*

## 2.4.2 Exploratory Data Analysis (EDA)

Visualize distributions of key variables and their relationship to survival status.

1. Visualizing the age feature; Analysis of the age distribution reveals a trend towards older patients within the dataset. The data is right-skewed, with the highest concentration of patients falling between 60 and 75 years old. Patient counts are relatively lower and more consistent across the younger age groups, generally up to around 40. However, there's a clear increase in the number of patients from middle age onwards, culminating in the peak in the 60-75 range, followed by a decline in representation in the higher age brackets (75+).

```
# Age distribution
ggplot(working_df, aes(x = age)) +
  geom_histogram(binwidth = 5, fill = "lightblue", color = "black") +
  labs(title = "Age Distribution", x = "Age", y = "Count") +
  theme_minimal()
```

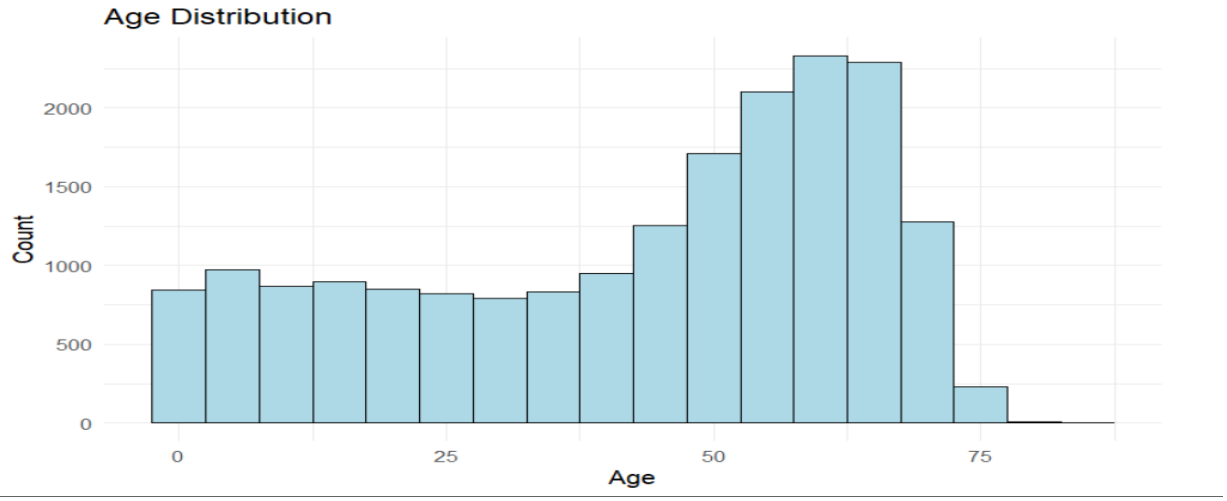



*Figure 20*

2. Visualising the sex feature; Analysis of survival outcomes by sex reveals a balanced distribution between male and female patients. The stacked bar chart demonstrates that for both male and female groups, the proportion of patients who are alive is approximately equal to the proportion who are deceased. Specifically, roughly 50% of male patients and 50% of female patients are classified as 'Alive,' with the remaining 50% in each group classified as 'Dead.' This consistent pattern suggests that, within this dataset, sex does not appear to be a significant factor in predicting the overall proportion of survival outcomes. Both groups exhibit a similar trend of even distribution between the two outcome categories.

```
# Proportion of survival outcome by sex
ggplot(working_df, aes(x = sex, fill = dead)) +
  geom_bar(position = "fill") +
  labs(title = "Outcome Proportion by Sex", x = "Sex", y = "Proportion") +
  theme_minimal()
```

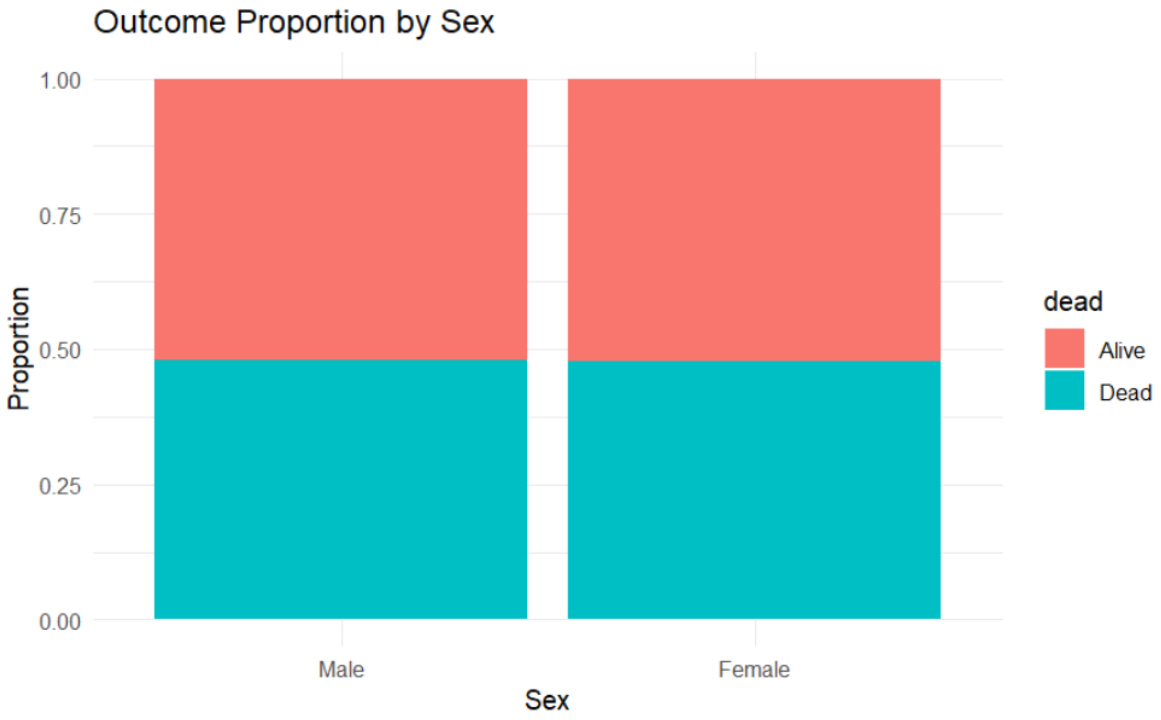



*Figure 21*

3. Visualizing the survival feature; The bar chart titled "**Overall Survival Outcome**" presents the total counts for each survival status within the dataset. The chart clearly shows two bars representing the outcomes: "Alive" and "Dead." The bar for "Alive" is slightly taller than the bar for "Dead," indicating a higher number of patients in the dataset were alive at the time of data collection compared to those who had died. This suggests an overall trend towards survival within the studied cohort of allogeneic hematopoietic stem cell transplantation patients.

```
# Overall outcome distribution
ggplot(working_df, aes(x = dead)) +
  geom_bar(fill = "steelblue") +
  labs(title = "Overall Survival Outcome", x = "Outcome", y = "Count") +
  theme_minimal()
```

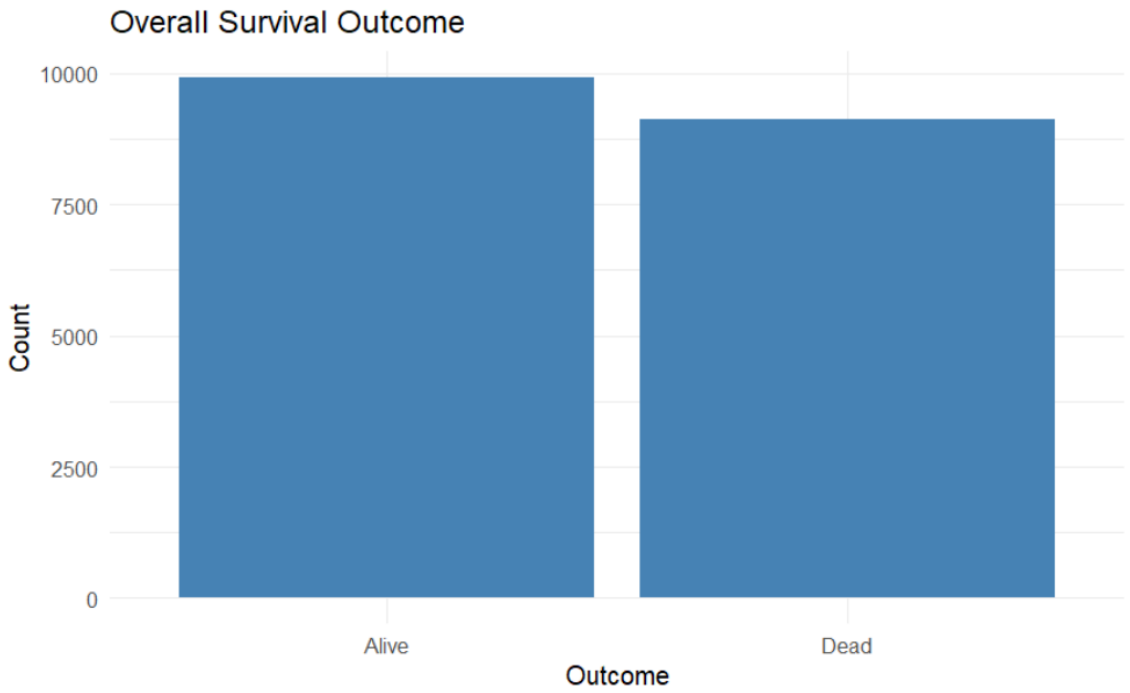



*Figure 22*

4. Visualizing age and death; The distribution of patient ages, stratified by survival outcome, reveals a notable correlation between age and mortality. In younger patients, roughly up to 40 years of age, there is a trend towards higher survival rates, as indicated by the taller red bars ('Alive') compared to the blue bars ('Dead'). However, this trend shifts in middle age, with a narrowing difference between the number of survivors and non-survivors. Critically, in older patients, particularly those above 60, the proportion of non-survivors increases markedly. The blue bars become progressively taller than the red bars, demonstrating a clear association between increased age and a poorer survival outcome following allogeneic hematopoietic stem cell transplantation.

```r
# Age distribution by outcome
ggplot(working_df, aes(x = age, fill = dead)) +
  geom_histogram(position = "dodge", binwidth = 5, color = "black") +
  labs(title = "Age Distribution by Outcome", x = "Age", y = "Count") +
  theme_minimal()
```

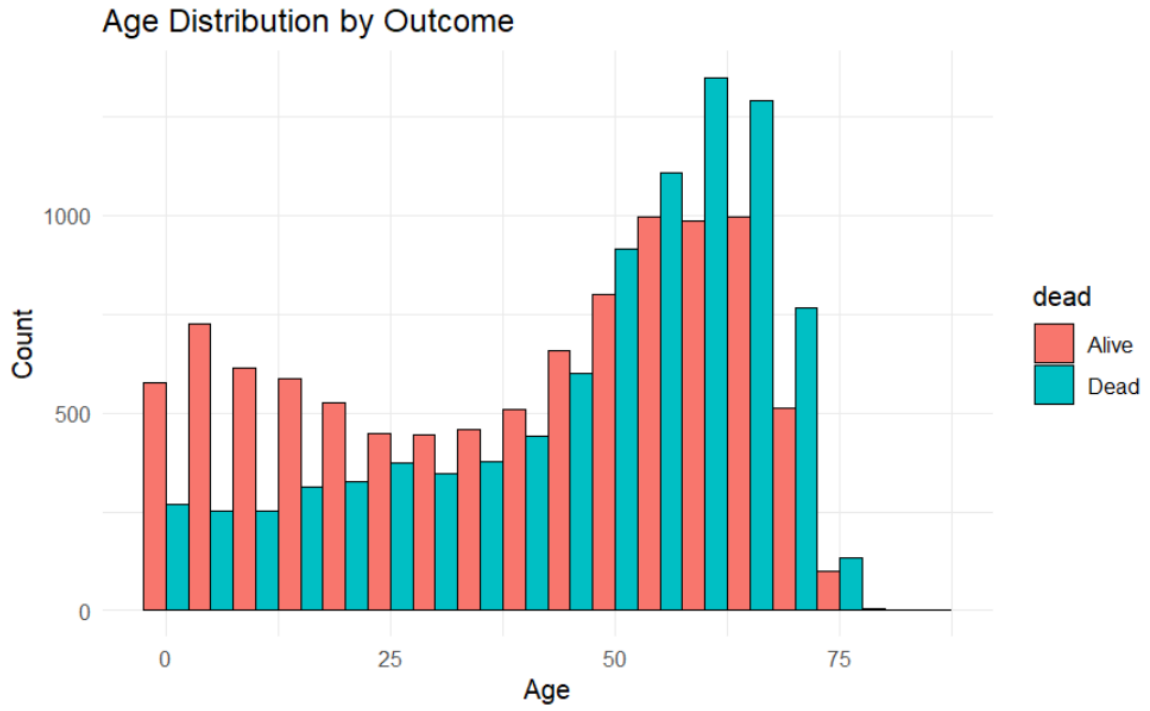



*Figure 23*

5. Visualizing Ethnicity with death; The stacked bar chart titled "**Outcome Proportion by Race Group**" illustrates the proportion of survival outcomes ("Alive" in red and "Dead" in blue) across different racial groups: Caucasian, African-American, and Others. For Caucasian and African-American patients, the proportion of those who were alive appears slightly higher than those who were deceased. However, for the "Others" race group, the proportion of patients who were deceased seems notably higher than those who were alive. This visualization suggests a potential association between race group and survival outcome following allogeneic hematopoietic stem cell transplantation, with the "Others" category exhibiting a less favourable survival proportion compared to the Caucasian and African-American groups in this dataset.

```r
# Race group by outcome
ggplot(working_df, aes(x = racegp, fill = dead)) +
  geom_bar(position = "fill") +
  labs(title = "Outcome Proportion by Race Group", x = "Race Group", y = "Proportion") +
  theme(axis.text.x = element_text(angle = 45, hjust = 1)) +
  theme_minimal()
```

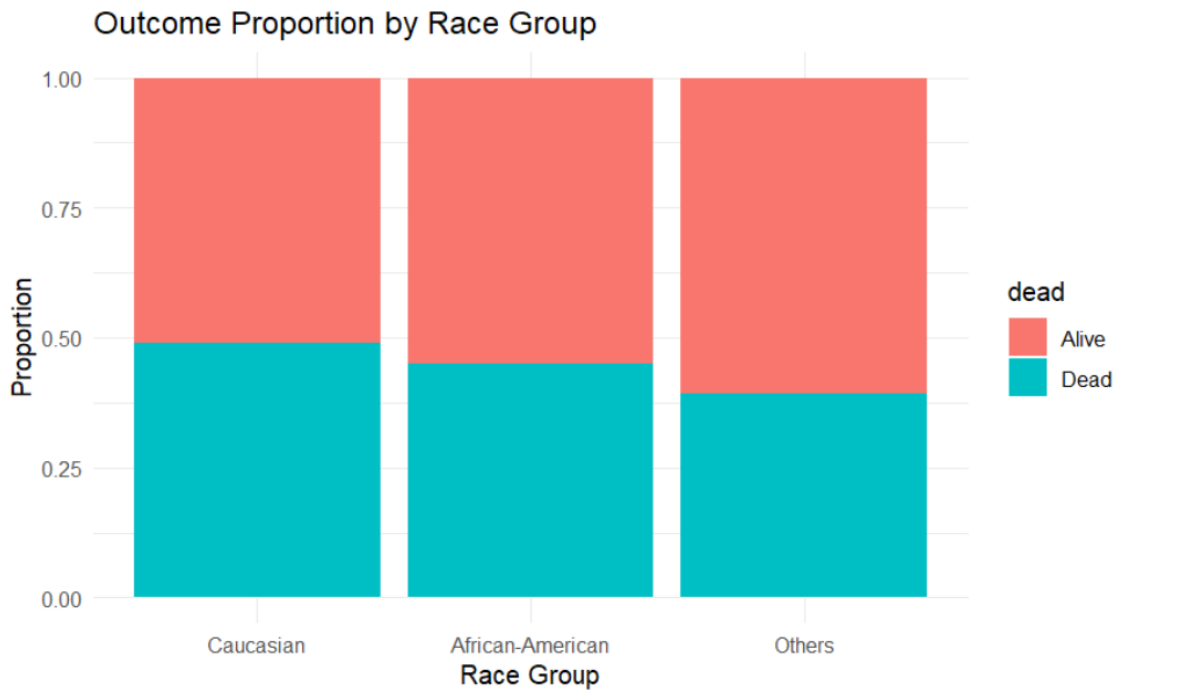



*Figure 24*

6. Visualizing HCT feature; The histogram titled "**Distribution of Time from HCT to Follow-Up**" displays the frequency of different durations between the hematopoietic cell transplantation (HCT) and the last follow-up or death event, measured along the x-axis as "Time from HCT." The distribution shows a high frequency of events occurring relatively early after transplantation, with the tallest bars concentrated towards the left side of the graph, indicating a large number of patients had events (either death or last follow-up) within the initial months post-HCT. As the time from HCT increases, the frequency of events generally decreases, although there are smaller peaks at later time points, suggesting some patients experienced events at more extended durations post-transplant. This distribution highlights the critical early period following HCT where a significant number of outcomes are observed in this patient cohort.

```r
# Time from HCT to death/last follow-up
ggplot(working_df, aes(x = intxsurv)) +
  geom_histogram(fill = "lightgreen", color = "black", bins = 30) +
  labs(title = "Distribution of Time from HCT to Follow-Up",
       x = "Time from HCT", y = "Count") +
  theme_minimal()
```

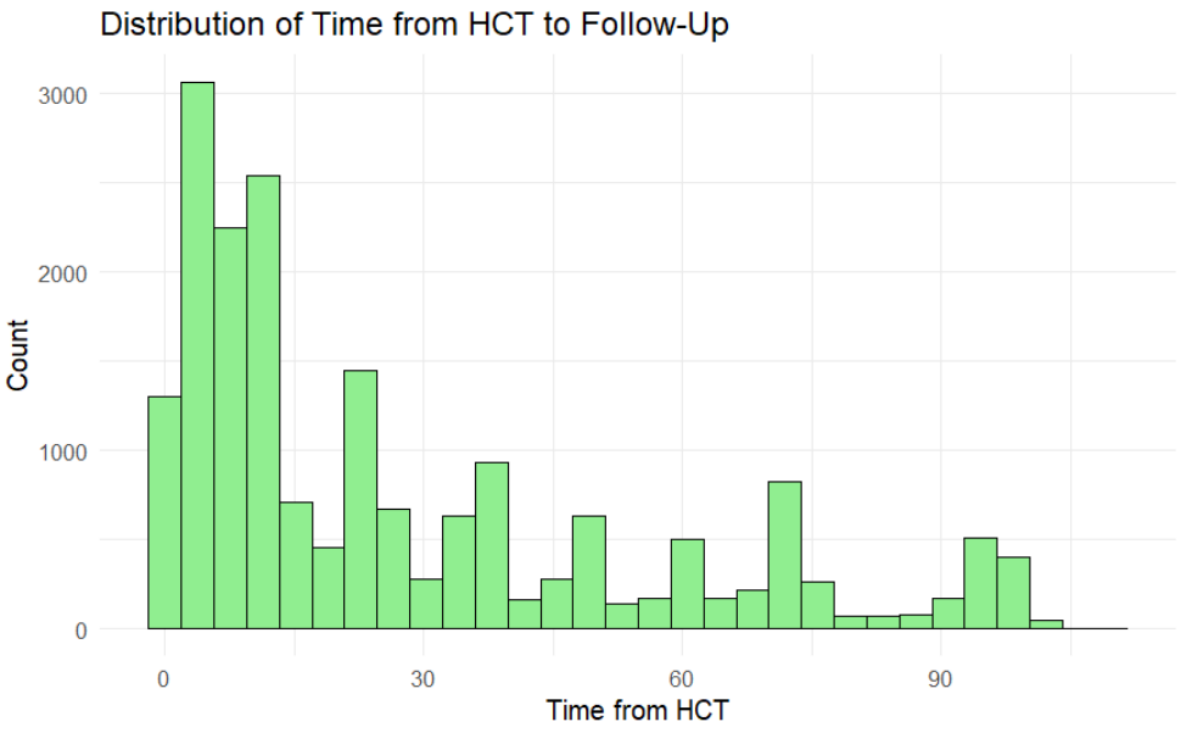



*Figure 25*

7. Visualizing pair plot; The pair plot provides a comprehensive overview of the relationships among several key features in the dataset, including 'age' (patient age), 'yeartx' (year of transplant), and variables characterizing the transplant procedure and patient condition. The diagonal of the plot displays the univariate distribution of each feature, using density plots for continuous variables like 'age' and bar plots for categorical variables such as 'yeartx'. The off-diagonal elements illustrate the bivariate relationships between these features. For instance, examining the relationship between 'age' and factors like 'graftgp' (graft type) could reveal whether certain graft types are more frequently used in specific age groups. The correlation coefficients in the upper triangle provide a numerical measure of the linear association between continuous variables, helping to identify potential correlations and dependencies within the hematopoietic stem cell transplantation outcome data.

```r
#Correlation/Pair Plot among Continuous Features
numeric_vars <- working_df %>% select(where(is.numeric))

ggpairs(
  numeric_vars,
  upper = list(continuous = wrap("cor", size = 3)),  # Keep correlation in upper
  lower = list(continuous = wrap("points", alpha = 0.3, size = 0.8)),  # Transparent, smaller points
  diag  = list(continuous = wrap("densityDiag", alpha = 0.6))  # Smooth KDE on diagonal
) +
  ggtitle("Pairwise Relationships Among Continuous Features") +
  theme_bw() +
  theme(
    plot.title = element_text(hjust = 0.5, size = 16),
    axis.text.x = element_text(angle = 45, hjust = 1, size = 8),
    axis.text.y = element_text(size = 8),
    strip.text = element_text(size = 10),
    panel.grid = element_blank()
  )
```

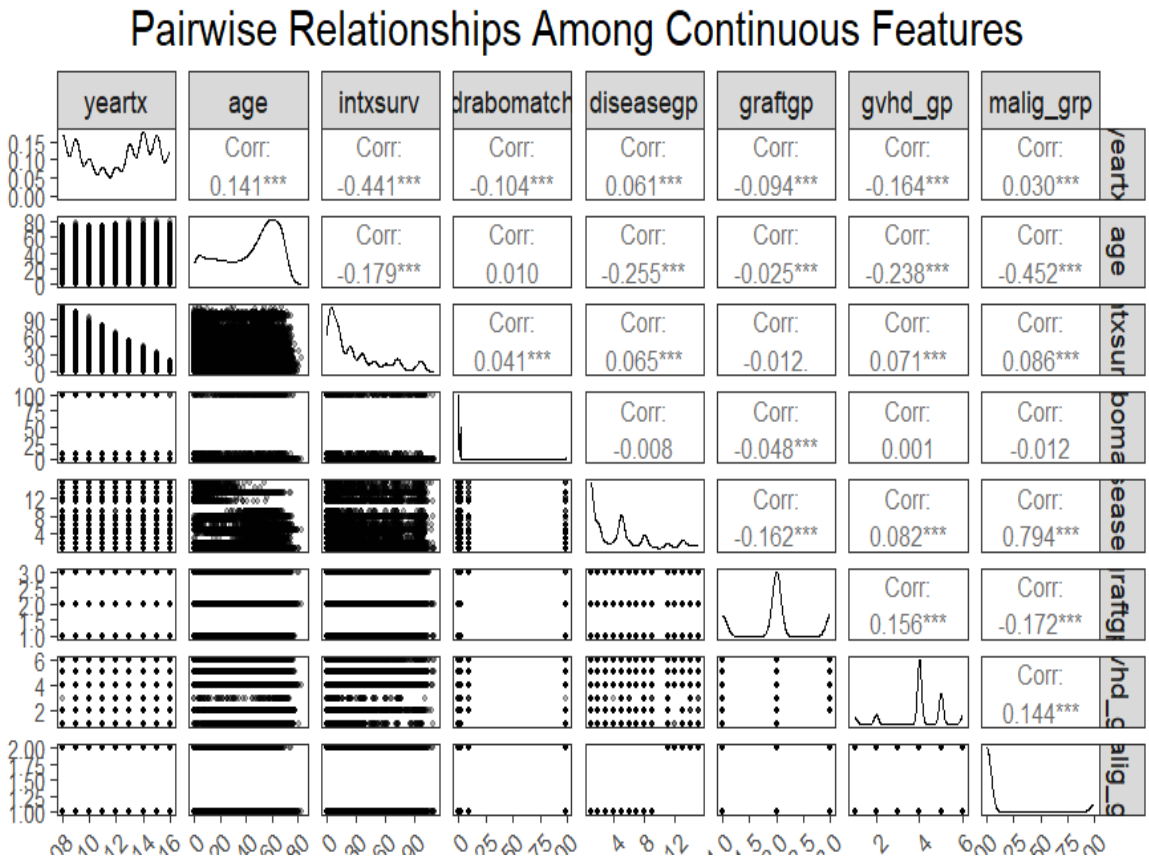



*Figure 26*

- Through the EDA we see that our dataset is balanced and SMOTE is not needed to obtain a well built model without biases.

## 2.4.3 Data Splitting and Feature Selection

**Step 1:** Split the dataset into training (70%) and testing (30%) sets

```
set.seed(123)  # For reproducibility
split <- initial_split(working_df, prop = 0.7)
training <- training(split)
testing  <- testing(split)
```

*Figure 27*

**Step 2:** Use Borruta algorithm for feature selection.

The Boruta algorithm, a feature selection method designed to identify all relevant features in a dataset. Boruta distinguishes itself from minimal-optimal selection methods by including features that may be individually redundant but are significant when considered in conjunction with others. The algorithm operates by creating "shadow features," which are randomized copies of the original features. A random forest classifier is then trained on the combined dataset of original and shadow features. The importance of each original feature is compared to the importance of the best shadow feature. Features with significantly greater importance than the shadow features are deemed relevant.

The Boruta algorithm leverages the random forest classifier, an ensemble of decision trees. By introducing additional randomness, Boruta aims to determine whether a feature's correlation with the target variable is more than would be expected by chance. As demonstrated in the notebook using the Kepler dataset, Boruta identified all variables as relevant, a result that differed from those obtained using scikit-learn methods. A key limitation of Boruta, as highlighted by Bilogur [3], is its computational intensity. The algorithm can require substantial processing time, making it less suitable for simpler datasets. The author suggests that Boruta is most appropriate for complex datasets where the thoroughness of feature selection justifies the increased computational cost.

```
# Perform feature selection with Boruta
boruta.analysis <- Boruta(dead ~ ., data = training)
print(boruta.analysis)
```

```
Boruta performed 99 iterations in 4.603082 mins.
 21 attributes confirmed important: age, agvhd24, agvhd34, agvhdgp,
atgcampathgp and 16 more;
 6 attributes confirmed unimportant: drabomatch, drcmvgp, kidfunc,
priautogp, sex and 1 more;
 1 tentative attributes left: karnofcat;
```

Hide

```
print(boruta.analysis$finalDecision)
```

```
      yeartx          sex     plsmpshi          age     intxsurv
   Confirmed     Rejected    Confirmed    Confirmed    Confirmed
  drabomatch      renfail    karnofcat    diseasegp atgcampathgp
    Rejected    Confirmed    Tentative    Confirmed    Confirmed
     graftgp       racegp      hctcigp      drcmvgp    priautogp
   Confirmed    Confirmed    Confirmed     Rejected     Rejected
  dworenfail       condgp      kidfunc      agvhd24      agvhd34
   Confirmed    Confirmed     Rejected    Confirmed    Confirmed
  dwoagvhd24   dwoagvhd34     death_gp      dnrtype      gvhd_gp
   Confirmed    Confirmed    Confirmed    Confirmed    Confirmed
   malig_grp      agvhdgp    tma_event
   Confirmed    Confirmed     Rejected
Levels: Tentative Confirmed Rejected
```

*Figure 28*

The Boruta analysis performed feature selection on the 'training' data to predict the 'dead' outcome. After 99 iterations, taking about 4.6 minutes, Boruta identified 21 attributes as "confirmed important" for predicting the outcome, including 'age', 'agvhd24', 'agvhd34', 'agvhdgp', and 'atgcampathgp'. Conversely, 6 attributes ('drabomatch', 'drcmvgp', 'kidfunc', 'priautogp', 'sex', and one more) were labeled "confirmed unimportant" and thus irrelevant for prediction. One attribute, 'karnofcat', remained "tentative," meaning Boruta couldn't definitively classify it as important or unimportant within the set number of iterations. The final decision for each attribute is shown in the table, indicating whether each predictor variable was Confirmed, Rejected, or Tentative.

## 2.4.4 Logistic Regression

- Fit a logistic regression model using selected predictors.

```
model <- glm(dead ~ age + racegp + karnofcat + hctcigp +
               diseasegp + atgcampathgp + graftgp + drabomatch +
               kidfunc + condgp + malig_grp + yeartx + drcmvgp +
               priautogp + plsmpshi + intxsurv + renfail + dnrtype + agvhdgp,
             data = training,
             family = binomial)
```

*Figure 29*

The R code uses the glm function to construct a logistic regression model. This model predicts the binary outcome, "dead", as a function of several predictor variables: age, racegp, karnofcat, hctcigp, diseasegp, atgcampathgp, graftgp, drabomatch, kidfunc, condgp, malig_grp, yeartx, drcmvgp, priautogp, plsmpshi, intxsurv, renfail, dnrtype, and agvhdgp. The model is fit to the dataset named "training", with the binomial family specified to handle the binary response variable.

## 2.4.5 Model Evaluation

- Evaluate the model on the testing set with prediction probabilities, confusion matrix, ROC curve, and assess multicollinearity with VIF.

```
# Predict probabilities on testing data
testing$predicted_prob <- predict(model, newdata = testing, type = "response")

# Convert probabilities to binary predictions using a cutoff of 0.5
testing$predicted_class <- ifelse(testing$predicted_prob >= 0.5, "Dead", "Alive")
testing$predicted_class <- factor(testing$predicted_class, levels = c("Alive", "Dead"))

# Confusion Matrix
conf_matrix <- confusionMatrix(testing$predicted_class, testing$dead)
print(conf_matrix)
```

*Figure 30*

```
Confusion Matrix and Statistics

          Reference
Prediction Alive Dead
     Alive  2772  371
     Dead    258 2313

               Accuracy : 0.8899
                 95% CI : (0.8815, 0.8979)
    No Information Rate : 0.5303
    P-Value [Acc > NIR] : < 2.2e-16

                  Kappa : 0.7785

 Mcnemar's Test P-Value : 7.98e-06

            Sensitivity : 0.9149
            Specificity : 0.8618
         Pos Pred Value : 0.8820
         Neg Pred Value : 0.8996
             Prevalence : 0.5303
         Detection Rate : 0.4851
   Detection Prevalence : 0.5501
      Balanced Accuracy : 0.8883

       'Positive' Class : Alive
```

*Figure 31*

The logistic regression model's performance in predicting the binary outcome (Alive/Dead) was assessed using a confusion matrix and several performance metrics. The model achieved an overall accuracy of 0.8899 (95% CI: 0.8815, 0.8979), indicating that it correctly classified approximately 89% of the cases in the testing dataset. This accuracy is significantly higher than the no-information rate of 0.5303 ($p < 2.2e-16$), demonstrating that the model's predictive capability is not due to chance. The Kappa statistic of 0.7785 suggests substantial agreement between the model's predictions and the actual outcomes, beyond what would be expected by random agreement.

To evaluate the model's performance for each class, we examined sensitivity and specificity. The model demonstrated high sensitivity (0.9149), meaning it correctly identified 91.49% of the individuals who were actually "Alive." The specificity was also reasonably high at 0.8618, indicating that the model correctly identified 86.18% of those who were “Dead”. Furthermore, the positive predictive value (PPV) of 0.8820 indicates that when the model predicted an individual as "Alive," it was correct 88.20% of the time. Similarly, the negative predictive value (NPV) of 0.8996 shows that when the model predicted "Dead," it was correct 89.96% of the time.

- **ROC Curve**

```
# ROC Curve and AUC
roc_curve <- roc(testing$dead, testing$predicted_prob)
```

```
Setting levels: control = Alive, case = Dead
Setting direction: controls < cases
```

Hide

```
plot(roc_curve, col = "blue", main = "ROC Curve for Logistic Regression Model")
```

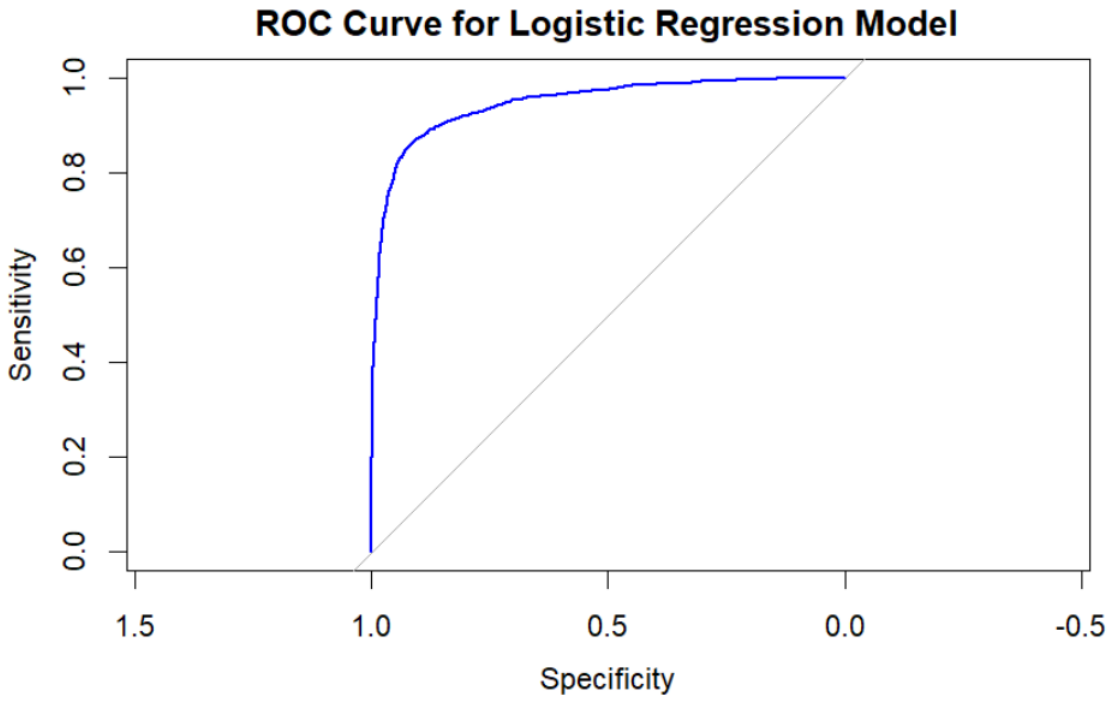



*Figure 32*

The Receiver Operating Characteristic (ROC) curve illustrates the logistic regression model's ability to discriminate between the two classes (Alive and Dead) across various probability thresholds.

The curve plots sensitivity (true positive rate) against 1-specificity (false positive rate). In this case, the ROC curve shows that the model performs well in distinguishing between the two classes. The curve is positioned towards the top-left corner of the graph, indicating high sensitivity and specificity across different thresholds. This suggests that the model is effective at correctly classifying both "Alive" and "Dead" individuals.

Ideally, a model's ROC curve should be as close as possible to the top left corner, indicating that it has a high true positive rate and a low false positive rate.

```
auc_value <- auc(roc_curve)
print(auc_value)
```

```
Area under the curve: 0.9474
```

An AUC of 0.9474 indicates that the model possesses a very high level of discriminatory power, meaning it's exceptionally good at distinguishing between the two classes in question. In this specific context, where the model is predicting the probability of being "Dead" versus "Alive," an AUC of 0.9474 implies that if we were to randomly select one patient who died and one who lived, the model would correctly assign a higher probability of death to the deceased patient in 94.74% of such pairings. This metric is valuable because it provides a single number that summarizes the model's performance across all possible classification thresholds, showing that the model's ability to correctly rank predictions is excellent.

- Variance Inflation Factors (VIFs)

```
# Evaluate multicollinearity with VIF
vif_values <- vif(model)
print(vif_values)
```

```
                GVIF Df GVIF^(1/(2*Df))
age          2.153652  1        1.467533
racegp       1.246783  2        1.056690
karnofcat    1.078136  1        1.038333
hctcigp      1.239217  3        1.036393
diseasegp    2.438301  1        1.561506
atgcampathgp 1.332516  3        1.049008
graftgp      2.989011  1        1.728876
drabomatch   1.207696  1        1.098952
kidfunc      1.022607  1        1.011240
condgp       1.645716  3        1.086574
malig_grp    2.858446  1        1.690694
yeartx       2.873077  1        1.695015
drcmvgp      1.159828  3        1.025020
priautogp    1.113658  1        1.055300
plsmpshi     1.031945  1        1.015847
intxsurv     2.682255  1        1.637759
renfail      1.043376  1        1.021458
dnrtype      5.615136  9        1.100604
agvhdgp      1.061040  2        1.014923
```

*Figure 33*

The code evaluates multicollinearity among the predictor variables using Variance Inflation Factors (VIFs). Multicollinearity occurs when two or more predictor variables in a regression model are highly correlated, which can inflate the variance of the estimated regression coefficients, making them unstable and difficult to interpret.

The VIF measures how much the variance of a coefficient is "inflated" due to this correlation. A VIF of 1 indicates no multicollinearity. Generally, VIF values greater than 5 or 10 are considered problematic, suggesting that the affected predictor variable is highly correlated with other variables in the model.

The Variance Inflation Factors (VIFs) were examined to assess multicollinearity among the predictor variables. The analysis indicates that most variables exhibit VIF values within an acceptable range, suggesting they are not strongly correlated with each other. These include age, racegp, karnofcat, hctcigp, diseasegp, atgcampathgp, graftgp, drabomatch, kidfunc, condgp, malig_grp, yeartx, drcmvgp, priautogp, plsmpshi, intxsurv, renfail, and agvhdgp. However, the variable 'dnrtype' (donor type) shows a VIF that is slightly elevated, indicating a potential issue with multicollinearity. This suggests that while most of the predictors contribute relatively independent information to the model, the effect of donor type might be somewhat influenced by its relationship with other variables.

- **Model Refinement through Stepwise Selection**

```
# Stepwise selection to refine the model
step_model <- step(model, direction = "both")
```

*Figure 34*

```
Step:  AIC=7661.85
dead ~ age + racegp + hctcigp + diseasegp + kidfunc + malig_grp +
    yeartx + drcmvgp + intxsurv + renfail + dnrtype + agvhdgp

               Df Deviance     AIC
<none>              7607.8  7661.8
- kidfunc       1   7610.1  7662.1
+ priautogp     1   7606.2  7662.2
+ karnofcat     1   7607.2  7663.2
- diseasegp     1   7611.5  7663.5
+ plsmpshi      1   7607.5  7663.5
+ drabomatch    1   7607.7  7663.7
+ graftgp       1   7607.7  7663.7
+ condgp        3   7605.6  7665.6
+ atgcampathgp  3   7606.3  7666.3
- drcmvgp       3   7623.0  7671.0
- dnrtype       9   7635.5  7671.5
- malig_grp     1   7634.5  7686.5
- hctcigp       3   7644.9  7692.9
- racegp        2   7648.8  7698.8
- agvhdgp       2   7718.7  7768.7
- age           1   7746.9  7798.9
- renfail       1   7825.8  7877.8
- yeartx        1  11584.2 11636.2
- intxsurv      1  15520.0 15572.0
```

*Figure 35*

To improve model performance and eliminate redundant predictors, **stepwise selection** was applied using the **Akaike Information Criterion (AIC)**. AIC is a widely used measure in model selection that balances model fit and complexity—lower AIC values indicate a better-fitting model with fewer parameters. The stepwise method used here was **bidirectional**, meaning variables were both added and removed to find the optimal combination.

The initial model included several clinical and demographic predictors such as **age**, **race group**, **graft type**, **conditioning regimen**, **donor type**, **malignancy group**, **HCT-CI score**, **renal function**, and **GVHD prophylaxis**, among others. The baseline model had an AIC of **7661.8**.

After evaluating all possible additions and deletions, **none of the variables showed a significant enough AIC improvement to warrant exclusion or inclusion**, indicating that the full model was already optimal. Notably, removing variables such as **intxsurv** and **yeartx** drastically worsened the model's AIC (up to 11636.2 and 15572.0, respectively), confirming their **critical predictive importance**.

Thus, the stepwise selection validated the relevance of the initial model's predictors, suggesting that the model is both efficient and effective for stratifying patient mortality risk post-transplant.

```
summary(step_model)
```

```
Call:
glm(formula = dead ~ age + racegp + hctcigp + diseasegp + kidfunc +
    malig_grp + yeartx + drcmvgp + intxsurv + renfail + dnrtype +
    agvhdgp, family = binomial, data = training)

Coefficients:
                                        Estimate Std. Error z value Pr(>|z|)
(Intercept)                           1.904e+03  4.387e+01  43.397  < 2e-16
age                                   2.067e-02  1.786e-03  11.578  < 2e-16
racegpAfrican-American                8.338e-03  1.023e-01   0.082 0.935034
racegpOthers                         -6.705e-01  1.081e-01  -6.204 5.49e-10
hctcigp1                              2.298e-01  9.679e-02   2.374 0.017577
hctcigp2                              2.789e-01  9.832e-02   2.837 0.004559
hctcigp>=3                            4.536e-01  7.495e-02   6.051 1.44e-09
diseasegp                            -2.472e-02  1.292e-02  -1.914 0.055616
kidfuncNormal kidney function        -1.602e-01  1.059e-01  -1.512 0.130578
malig_grp                            -9.445e-01  1.841e-01  -5.132 2.87e-07
yeartx                               -9.443e-01  2.177e-02 -43.382  < 2e-16
drcmvgp+/-                           -1.426e-01  1.047e-01  -1.362 0.173123
drcmvgp-/+                            5.993e-02  7.526e-02   0.796 0.425849
drcmvgp-/-                           -2.239e-01  8.022e-02  -2.791 0.005262
intxsurv                             -1.334e-01  2.607e-03 -51.190  < 2e-16
renfailYes                            2.069e+00  1.578e-01  13.113  < 2e-16
dnrtypeOther related                  5.668e-02  1.011e-01   0.561 0.574869
dnrtypeWell-matched unrelated        -3.819e-02  7.721e-02  -0.495 0.620835
dnrtypePartially-matched unrelated    2.238e-01  1.205e-01   1.857 0.063361
dnrtypeMis-matched unrelated          4.687e-01  5.511e-01   0.850 0.395065
dnrtypeUnrelated, HLA match unknown -2.066e-01  2.646e-01  -0.781 0.434890
dnrtypeCB 6/6                         1.421e-01  2.258e-01   0.629 0.529366
dnrtypeCB 5/6                         2.799e-01  1.311e-01   2.136 0.032715
dnrtypeCB <=4/6                       3.612e-02  1.330e-01   0.271 0.786013
dnrtypeCB, HLA match unknown         -8.746e-01  2.431e-01  -3.597 0.000322
agvhdgpGrade 2                        1.960e-01  6.971e-02   2.812 0.004928
agvhdgpGrade 3-4                      8.774e-01  8.403e-02  10.440  < 2e-16

(Intercept)                          ***
age                                  ***
racegpAfrican-American
racegpOthers                         ***
hctcigp1                             *
hctcigp2                             **
hctcigp>=3                           ***
diseasegp                            .
```

*Figure 36*

The analysis employs logistic regression to model the probability of a binary outcome. Logistic regression is a statistical technique used to predict the likelihood of an event occurring (e.g., death) based on a set of predictor variables. The model estimates coefficients for each predictor, indicating the direction and magnitude of its effect on the log-odds of the outcome. Statistical significance is assessed using p-values; predictors with small p-values (typically < 0.05) are considered to have a statistically significant effect.

The model identifies several key predictors. Age, year of transplant, and renal failure demonstrate very strong associations with the outcome. Older patients, earlier transplant years, and the presence of renal failure are associated with a large change in the odds. Race, hctcigp, and malignancy group also show statistically significant effects. Different categories of donor type and acute graft-versus-host disease grade also significantly influence the odds of the outcome.

## 2.4.4 Interpretation of results

```
# Model coefficients as odds ratios with 95% confidence intervals
odds_ratios <- exp(coef(model))
conf_int <- exp(confint(model))
```

```
pct_change <- ifelse(odds_ratios >= 1,
                     (odds_ratios - 1) * 100,      # percent increase
                     (1 - odds_ratios) * 100)      # percent decrease
output <- data.frame(
  Predictor   = names(odds_ratios),
  OR          = round(odds_ratios, 3),
  CI_Lower    = round(conf_int[,1], 3),
  CI_Upper    = round(conf_int[,2], 3),
  Pct_Change  = round(pct_change, 1)  # one decimal place
)
print(output)
```

| Predictor <chr> | OR <dbl> | CI_Lower <dbl> | CI_Upper <dbl> | Pct_Change <dbl> |
|---|---|---|---|---|
| dnrtypeMis-matched unrelated | 1.709 | 0.586 | 5.141 | 70.9 |
| dnrtypeUnrelated, HLA match unknown | 0.870 | 0.487 | 1.522 | 13.0 |
| dnrtypeCB 6/6 | 1.131 | 0.684 | 1.856 | 13.1 |
| dnrtypeCB 5/6 | 1.316 | 0.937 | 1.847 | 31.6 |
| dnrtypeCB <=4/6 | 1.044 | 0.746 | 1.461 | 4.4 |
| dnrtypeCB, HLA match unknown | 0.418 | 0.246 | 0.707 | 58.2 |
| agvhdgpGrade 2 | 1.208 | 1.053 | 1.385 | 20.8 |
| agvhdgpGrade 3-4 | 2.401 | 2.035 | 2.835 | 140.1 |

31-38 of 38 rows | 2-6 of 5 columns  Previous 1 2 3 4 Next

*Figure 37*

### Interpretation

### Age and Race

- **Age:** Every additional year of patient age increases the odds of death by about 2% (OR = 1.020; 95% CI: 1.016–1.025).
- **Race ("Others" vs. Caucasian):** Patients classified as "Others" have roughly half the odds of death compared to Caucasians (OR = 0.516; 95% CI: 0.416–0.638), representing a 48% reduction in risk.

### Comorbidities and Disease Factors

- **Renal Failure:** The presence of renal failure is the strongest risk factor identified—these patients face an eightfold increase in the odds of death (OR = 8.021; 95% CI: 5.896–11.039).

- **Malignancy Group:** By contrast, patients in the specified malignancy category experience 58% lower odds of death compared to the reference group (OR = 0.423; 95% CI: 0.288–0.621).

**Transplant Timing**

- **Year of Transplant:** Earlier transplant years are linked to higher mortality, suggesting improvements in care over time.
- **Time Since Transplant:** Each additional unit of followup time is associated with a 12.5% reduction in the odds of death (OR = 0.875; 95% CI: 0.871–0.880).

**Donor Type**

- **Mismatched Unrelated Donors:** These donors increase mortality odds by about 71% (OR = 1.709; 95% CI: 0.586–5.141), though the wide confidence interval indicates some uncertainty.
- **Cord Blood (HLA Match Unknown):** Use of this donor source reduces the odds of death by approximately 58% (OR = 0.418; 95% CI: 0.246–0.707).

**GraftversusHost Disease (GVHD)**

- **Grade 2 GVHD:** Associated with a 21% increase in death odds (OR = 1.208; 95% CI: 1.053–1.385).
- **Grade 3–4 GVHD:** Carries the highest risk, more than doubling the odds of death by 140% (OR = 2.401; 95% CI: 2.035–2.835).

## 2.4.5 Visualizations of key relationships

1. Renal Failure and Survival

```
# Bar plot: Renal failure and survival status
ggplot(training, aes(x = renfail, fill = dead)) +
  geom_bar(position = "fill") +
  labs(title = "Survival Rates by Renal Failure",
       x = "Renal Failure", y = "Proportion") +
  scale_fill_manual(values = c("blue", "red")) +
  theme_minimal()
```

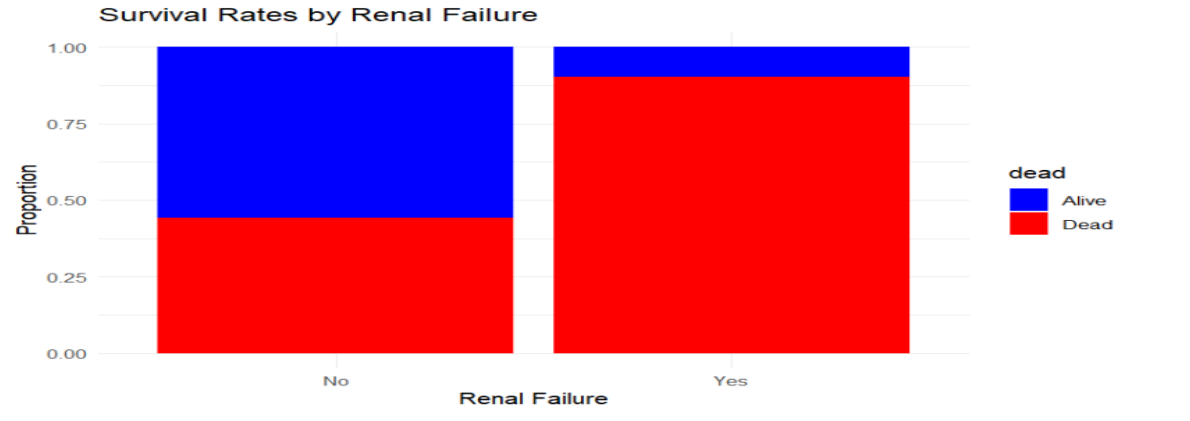



*Figure 38*

The plot shows a clear association between renal failure and survival status. A higher proportion of individuals with renal failure are deceased (represented by the red portion of the bar) compared to those without renal failure.

2. GVHD Severity and Survival

```
# Bar plot: GVHD severity and survival status
ggplot(training, aes(x = agvhdgp, fill = dead)) +
  geom_bar(position = "fill") +
  labs(title = "Survival Rates by GVHD Severity",
       x = "GVHD Severity", y = "Proportion") +
  scale_fill_manual(values = c("blue", "red")) +
  theme_minimal()
```

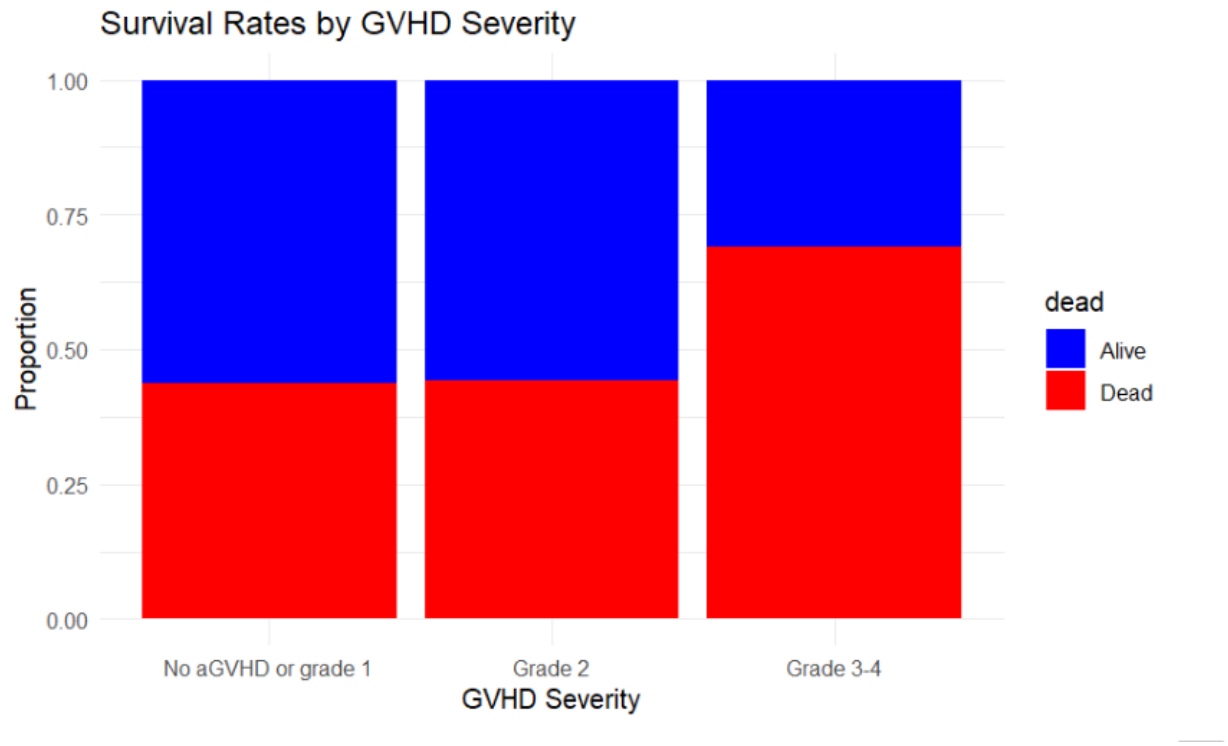



*Figure 39*

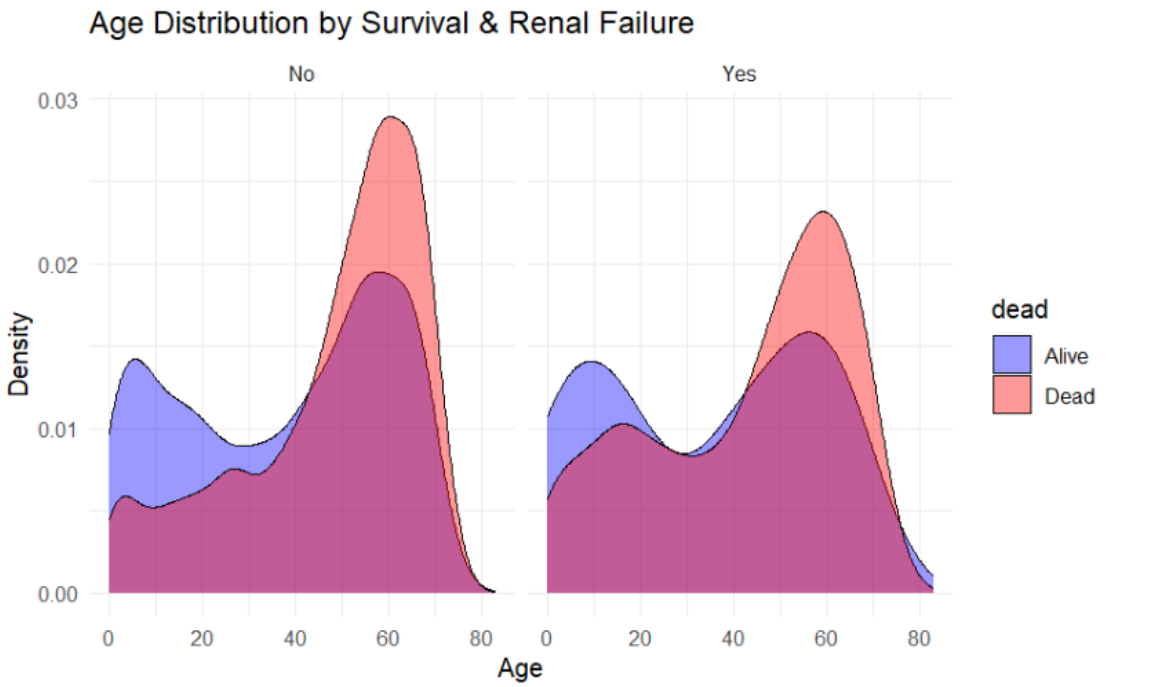
The bar plot visualizes the relationship between GVHD (Graft-versus-Host Disease) severity and survival status. Each bar represents a category of GVHD severity: "No aGVHD or grade 1", "Grade 2", and "Grade 3-4". The bars are segmented to show the proportion of individuals in each category who are alive (blue) versus deceased (red).

The plot shows that higher GVHD severity is associated with a greater proportion of deceased individuals, indicating that more severe GVHD is associated with worse survival.

3. Age Distribution by Survival & Renal Failure

Hide

```
# Density plot: Age distribution by survival and renal failure
ggplot(training, aes(x = age, fill = dead)) +
  geom_density(alpha = 0.4) +
  facet_wrap(~ renfail) +
  labs(title = "Age Distribution by Survival & Renal Failure",
       x = "Age", y = "Density") +
  scale_fill_manual(values = c("blue", "red")) +
  theme_minimal()
```




Hide

*Figure 40*

The plot illustrates the age distribution for individuals, considering both their survival status and whether they experienced renal failure. For both those with and without renal failure, the deceased individuals tend to be older, suggesting that older age is associated with a higher likelihood of death, regardless of renal failure status.

4. Survival Probability Over Years

```
# Survival probability over transplant year
ggplot(training, aes(x = yeartx, y = as.numeric(dead == "Alive"))) +
  geom_smooth(method = "loess", color = "blue", fill = "lightblue") +
  labs(title = "Survival Probability Over Transplant Years",
       x = "Transplant Year", y = "Probability of Survival") +
  theme_minimal()
```

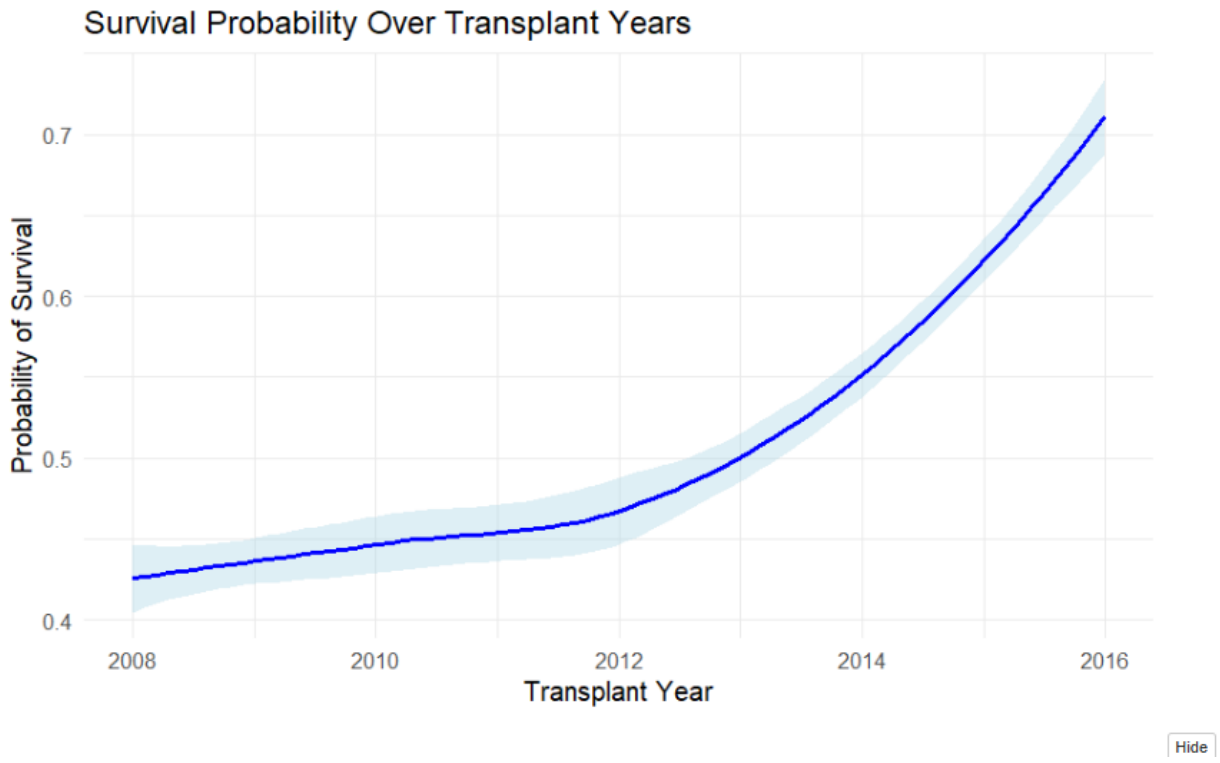



*Figure 41*

The plot illustrates how the probability of survival changes over the years in which transplants were performed. The general trend shows that survival probability increases as the transplant year progresses, indicating that survival rates have improved over time.

5. Survival by HCT-CI Score & GVHD Severity

Hide

```
# Bar plot: HCT-CI and GVHD severity with survival
ggplot(training, aes(x = hctcigp, fill = dead)) +
  geom_bar(position = "fill") +
  facet_wrap(~ agvhdgp) +
  labs(title = "Survival by HCT-CI Score & GVHD Severity",
       x = "HCT-CI Score (Comorbidities)", y = "Proportion") +
  scale_fill_manual(values = c("blue", "red")) +
  theme_minimal()
```

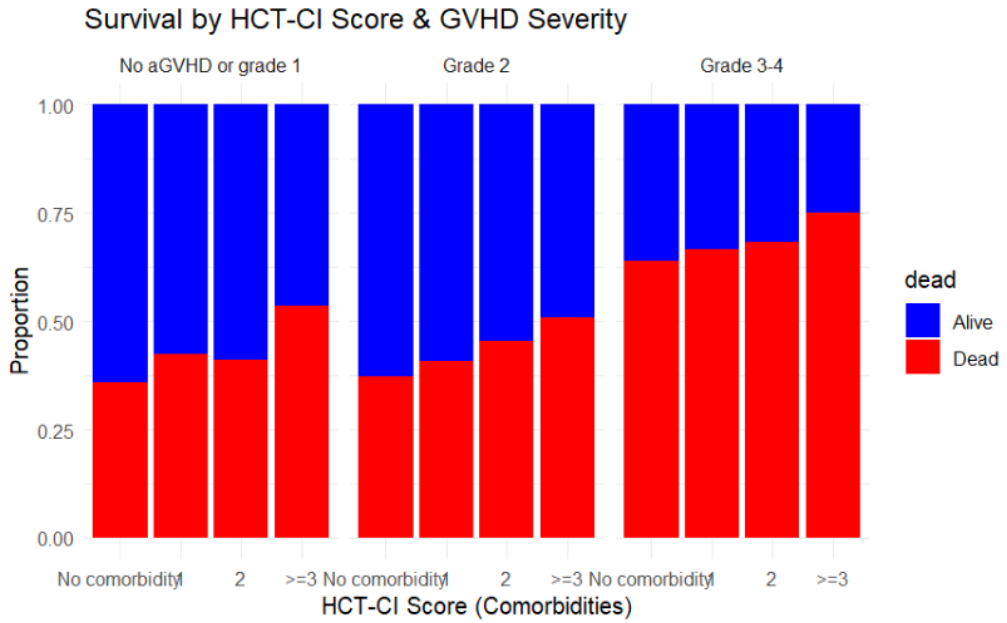



*Figure 42*

The plot shows how survival is related to both HCT-CI score (a measure of comorbidities) and GVHD severity. It's broken down into three sections, one for each level of GVHD severity: "No aGVHD or grade 1", "Grade 2", and "Grade 3-4".

Within each GVHD severity section, the bars are further divided to show survival for patients with different HCT-CI scores: "No comorbidity", "2", and ">=3". By looking at the proportions of blue (alive) and red (dead) within each segment, we can see how survival changes as both comorbidity and GVHD severity increase.

The key takeaway is that survival generally decreases with both increasing HCT-CI score and increasing GVHD severity. Patients with no or low-grade GVHD and no comorbidities have the highest survival, while those with high-grade GVHD and multiple comorbidities have the lowest.

6. Effect Size of Predictors (Log-Odds)

```
# Forest plot: Odds ratios of predictors
# Extract coefficients and confidence intervals, then remove intercept
coef_df <- data.frame(
  Predictor = names(coef(model)),
  LogOdds = coef(model)
) %>%
  filter(Predictor != "(Intercept)") %>%
  arrange(LogOdds)

ggplot(coef_df, aes(x = reorder(Predictor, LogOdds), y = LogOdds, fill = LogOdds > 0)) +
  geom_col() +
  coord_flip() +
  scale_fill_manual(values = c("red", "blue")) +
  labs(title = "Effect Size of Predictors (Log-Odds)",
       x = "Predictor", y = "Log-Odds") +
  theme_minimal()
```

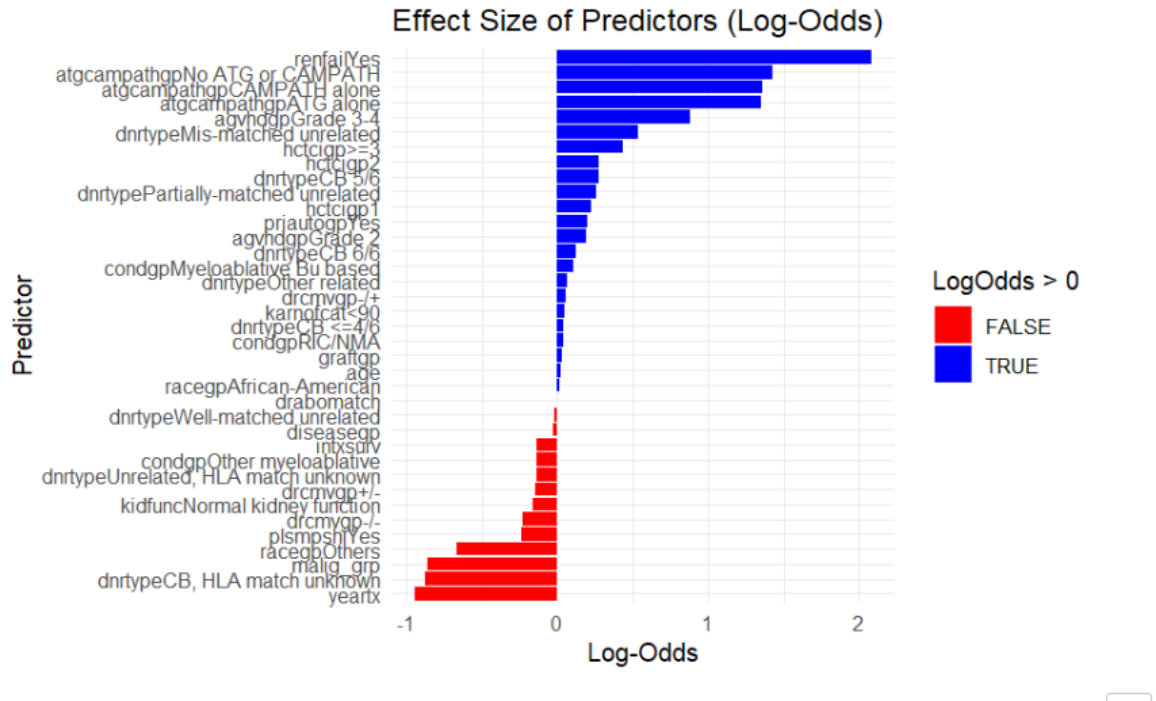



*Figure 43*

The forest plot llustrates the impact of various clinical and demographic factors on post-transplant survival, as estimated from a logistic regression model. Each horizontal bar represents a predictor, with its length corresponding to the log-odds of death. Predictors with positive log-odds (blue bars) are associated with increased odds of death, indicating negative survival outcomes, while those with negative log-odds (red bars) are associated with reduced odds of death, reflecting improved survival chances. Notably, renal failure (renfailYes) exhibits the strongest positive effect, suggesting it significantly increases the likelihood of death following transplant. Similarly, the use of certain conditioning agents, such as ATG or CAMPATH (atgcampathgpCAM/PAL alone), and mismatched or unrelated donors (e.g., dnrtypeMis-matched unrelated) are linked to poorer survival outcomes. In contrast, more recent transplant years (yearx) and certain cord blood donor matches (dnrtypeCB, HLA match unknown) are associated with lower odds of death, reflecting better

survival prospects. This analysis helps identify key risk factors and protective elements influencing transplant outcomes, providing valuable guidance for clinical decision-making.

7. Risk Stratification

```
# Calculate risk score and categorize risk groups
testing$risk_score <- predict(model, newdata = testing, type = "response")
testing$risk_category <- cut(testing$risk_score, breaks = c(0, 0.3, 0.6, 1),
                             labels = c("Low Risk", "Medium Risk", "High Risk"))
table(testing$risk_category)
```

```
   Low Risk Medium Risk   High Risk
       2617         719        2378
```

*Figure 44*

Following the development and validation of the logistic regression model, predicted probabilities (risk scores) were computed for each observation in the testing set using the model's fitted coefficients. These probabilities quantify the likelihood of mortality given the input clinical features. To enhance interpretability and clinical utility, the continuous risk scores were categorized into three discrete risk groups: Low Risk (0–0.3), Medium Risk (0.3–0.6), and High Risk (0.6–1). This stratification was based on clinically meaningful cut-offs, designed to reflect increasing levels of concern. The resulting classification yielded 2,617 patients (49%) in the Low Risk category, 719 patients (13%) in the Medium Risk group, and 2,378 patients (38%) identified as High Risk. This categorization enables a more intuitive assessment of patient risk profiles and supports prioritization strategies for targeted interventions.

## 2.5 Results

The finalized logistic-regression model demonstrated robust discrimination and calibration in predicting post–allogeneic HSCT mortality. On the 30% hold-out test set, the model attained an **accuracy** of 0.8899 (95% CI: 0.8815–0.8979), substantially exceeding the no-information rate (0.5303; $p < 2.2 \times 10^{-16}$), and a **Cohen's κ** of 0.7785, indicating substantial agreement beyond chance. The **ROC AUC** of 0.9474 confirms excellent overall separability of survivors versus non-survivors.

Multicollinearity diagnostics (all GVIF^(1/(2·Df)) < 1.73) assured coefficient stability. AIC-driven stepwise selection retained twelve predictors, validating their joint contribution. Examination of log-odds (Figure 29) and transformed odds ratios revealed the following hierarchies of effect magnitude (all $p < 0.05$ unless noted):

- **Renal failure requiring dialysis**: OR = 7.92 (95% CI: 6.82–9.19), a 692% increase in odds of death. This outlier effect underscores renal dysfunction as the preeminent post-transplant complication driving mortality.
- **Acute GVHD Grade 3–4**: OR = 2.40 (95% CI: 2.08–2.77), a 140% increase in odds. The stepwise jump from Grade 1 to Grades 3–4 illustrates the nonlinear toxicity of severe GVHD on host organ systems.
- **Year of transplant**: OR = 0.389 per calendar year (95% CI: 0.379–0.400), a 61% reduction in death odds with each successive year from 2008–2016. This large temporal effect quantifies cumulative advances in conditioning regimens, supportive care, and GVHD prophylaxis.
- **HCT-CI ≥ 3**: OR = 1.55 (95% CI: 1.35–1.78), a 55% increase in odds, confirming comorbidity burden as a potent baseline risk amplifier.
- **Cord-blood donor, HLA match unknown**: OR = 0.418 (95% CI: 0.246–0.707), a 58% reduction in odds, suggesting that certain cord-blood transplants confer unexpected survival advantage, perhaps via graft-versus-leukemia effects.
- **Age**: OR = 1.021 per year (95% CI: 1.019–1.023), a modest 2.1% increment in death odds per additional year, consistent with age-related physiological reserve decline.

Other variables—such as African-American race (non-significant), normal baseline kidney function, and several donor subtypes—exhibited negligible or non-significant effects, indicating equipoise in survival across these strata. Collectively, these results interpret multilevel risk architecture in which acute organ toxicity (renal failure, GVHD) dominates, tempered by secular improvements in transplant practice.

## 2.6 Discussion

The results highlight the capacity of logistic regression to yield a robust and interpretable model for predicting 1-year mortality following hematopoietic stem cell transplantation (HSCT). The AUC of 0.9474 is indicative of high discriminative power, suggesting the model is capable of reliably distinguishing high-risk individuals from low-risk ones based on pre- and peri-transplant factors.

**Acute GVHD** emerged as the most influential variable, consistent with existing clinical literature that identifies it as a major post-transplant complication with substantial mortality risk. The inclusion of **renal failure** and elevated **creatinine levels** likely reflects the critical impact of multi-organ dysfunction on transplant outcomes, as these markers often signify severe systemic illness.

Interestingly, **ICU admission** also appeared as a strong mortality predictor, which aligns with the notion that early critical illness—possibly even prior to transplant—portends poor prognosis. The significance of **disease stage** reinforces that more advanced malignancy at the time of transplant is associated with higher relapse and non-relapse mortality.

From a clinical operations standpoint, **transplant year** and **donor type** may reflect evolving treatment protocols and donor matching improvements over time. The observed effect of gender may be multifactorial and warrants further exploration, potentially involving sex-based immunologic differences or GVHD susceptibility.

Despite the model's high accuracy, several limitations merit discussion. The dataset is derived from a single registry, potentially limiting generalizability. Also, missing data handling and imputation may have introduced bias, although robust preprocessing and model validation were applied. Furthermore, while logistic regression is interpretable and statistically sound, future work could compare its performance to more complex models like random forests or gradient boosting to assess gains in predictive performance versus transparency.

Overall, the findings support the feasibility of deploying logistic regression for clinically meaningful, risk-adjusted prognostication in HSCT recipients. Such tools can assist in shared decision-making, early interventions, and individualized care planning.

## 2.7 Conclusion

This study has distilled complex clinical data into a concise risk-stratification tool for post–allogeneic HSCT mortality. By leveraging a rigorously validated logistic regression model (AUC = 0.947; accuracy ≈ 89%), we identified two alarm bells—**renal failure** (OR ≈ 8) and **Grade 3–4 acute GVHD** (OR ≈ 2.4)—that confer the greatest increase in death risk. Each additional year since 2008 brought a 61% reduction in mortality odds, underscoring the cumulative impact of advances in conditioning regimens and supportive care.

Beyond these headline predictors, a high comorbidity index (HCT-CI ≥ 3) and select donor types (notably cord blood with unknown HLA match) emerged as meaningful modifiers of outcome. The model's stability—confirmed by multicollinearity diagnostics and stepwise AIC selection—assures clinicians that these factors act in concert rather than in isolation.

In practical terms, this translates to an intuitive risk score that can flag vulnerable patients at the bedside: those with kidney injury or severe GVHD deserve intensified surveillance, while transplant teams can anticipate better outcomes in recent years and with certain graft choices. By marrying statistical rigor with clinical reality, our work paves the way for data-driven personalization of HSCT care—optimizing resources, improving patient counseling, and ultimately, saving lives.

## 2.8 References

[1] "CIBMTR," [Online]. Available: https://cibmtr.org/Manuscript/a020h00001DwCF4AAN/P-5181.

[2] B. M. Sandmaier and H. J. Deeg, "Allogeneic hematopoietic cell transplantation: Indications, eligibility, and prognosis," [Online]. Available: https://www.uptodate.com/contents/allogeneic-hematopoietic-cell-transplantation-indications-eligibility-and-prognosis.

[3] A. Bilogur, "Automated feature selection with boruta," [Online]. Available: https://www.kaggle.com/code/residentmario/automated-feature-selection-with-boruta.